Claude Multi-Account Fine Tuning

Running multiple Claude Code accounts on one host with shared history, memory, and settings

milos85vasic

2026-05-26

Claude Multi-Account Fine Tuning

This document captures, in reproducible detail, how a single host can run multiple Claude Code accounts (e.g. `claude1`, `claude2`, …, `claudeN`) while sharing conversation history, project memory, todos, plans, plugins, and settings between them. Only account identity (`.credentials.json`, `.claude.json`, MCP auth cache) is kept private per account.

Companion artifacts:

- `~/Documents/scripts/` — automation scripts (the source of truth for the procedure described here).
- `Claude_Multi_Account_Fine_Tuning.html` — same content as this file, rendered as a standalone HTML page.
- `Claude_Multi_Account_Fine_Tuning.pdf` — same content, paginated PDF.

The HTML and PDF are regenerated by `claude-export-docs.sh`, so editing the markdown is the only place changes need to be made.

---

1. Motivation

The Claude Code CLI is invoked as `claude`. By default it writes all of its state — credentials, conversation transcripts, project memory, todos, plans, plugin installs, settings — under a single config directory (`~/.claude` on Linux/macOS).

When you want to use **multiple Anthropic accounts** (for example, one personal Max subscription and one team/work account), you typically run two separate config dirs and switch between them with an environment variable:

```
# ~/.bashrc (before this setup)
export CLAUDE_BIN="$HOME/.local/bin/claude"
alias claude1="CLAUDE_CONFIG_DIR=$HOME/.claude-milos85vasic
        $CLAUDE_BIN"
alias claude2="CLAUDE_CONFIG_DIR=$HOME/.claude-milos85vasic2nd
        $CLAUDE_BIN"
```

That works for isolation, but the two accounts now have **completely separate** project memory, history, todos, and plugin sets. Switching accounts means losing the working context you built up under the other.

Goal: keep the two-alias UX (cheap account switch) while ensuring both accounts read and write the same project memory, conversation transcripts, todos, plans, plugins, and settings.

---

2. How Claude Code stores state

Every key piece of state lives under the directory pointed at by `CLAUDE_CONFIG_DIR` (with `~/.claude` as the default). Items fall into three categories:

Category	Path under `CLAUDE_CONFIG_DIR`	What it is
Identity (private)	`.credentials.json`	OAuth tokens for the Anthropic account.
	`.claude.json`	Account onboarding/ migration markers, numStartups, etc.
	`mcp-needs-auth-cache.json`	Per-account MCP auth state.
	`projects/<slug>/<uuid>.jsonl`	Per-session conversation transcript.

Category	Path under `CLAUDE_CONFIG_DIR`	What it is
Work product (share)		
	`projects/<slug>/memory/*.md`	Project memory entries (what `Skill auto-memory` writes).
	`history.jsonl`	Prompt-line history (`Up`/`Down` recall).
	`todos/`, `tasks/`, `plans/`	UUID-named files for TaskCreate/Plan tooling.
	`file-history/`	Snapshots of files edited via the Edit tool.
	`paste-cache/`	Long-paste buffer storage.
	`plugins/`	Installed plugins, marketplaces, manifest JSON.
	`settings.json`	Top-level config: `enabledPlugins`, `effortLevel`, etc.
	`CLAUDE.md`	User-scope memory / instructions.
Runtime (share-able with care)	`sessions/`	Active session lockfiles. **Not safe to share between concurrent processes.**
	`telemetry/`	Per-account telemetry batches.
	`cache/`, `backups/`, `session-env/`, `shell-snapshots/`	Ephemeral runtime caches.
	`stats-cache.json`	Per-account usage / quota stats.

The key observation: every **work-product** item is keyed either by UUID (session, todo, plan, file edit) or by project slug. That means combining two accounts' work-product directories is a **set-union operation**: copying both into the same destination never overwrites either's data, because each file's path is already unique.

The handful of files that genuinely need merging are:

- `settings.json` — both files have a top-level `enabledPlugins` map. We union it, then keep the most permissive top-level keys (`effortLevel: "high"`, `skipDangerousModePermissionPrompt: true`).
- `history.jsonl` — append-only prompt log. We concatenate both files and de-duplicate by line, preserving order of first appearance.
- `plugins/installed_plugins.json` and `plugins/known_marketplaces.json` — these embed **absolute paths** to plugin code. We rewrite those paths to live under the shared store regardless of which account they came from.

---

3. Architecture after fine-tuning

```
~/.claude-shared/                 <-- single source of truth
  projects/                       (transcripts + memory; per-slug)
  todos/  tasks/  plans/
  file-history/  paste-cache/
  plugins/
    installed_plugins.json        (paths rewritten -> ~/.claude-
shared/...)
    known_marketplaces.json       (paths rewritten -> ~/.claude-
shared/...)
    cache/<marketplace>/<plugin>/<version>/
    marketplaces/<marketplace>/
  shell-snapshots/  session-env/
  telemetry/  sessions/  backups/  cache/
  history.jsonl  settings.json  stats-cache.json
  CLAUDE.md

~/.claude-milos85vasic/           <-- account 1 (claude1 alias)
  .credentials.json    (PRIVATE — claude1's OAuth tokens)
  .claude.json         (PRIVATE — claude1's onboarding state)
  mcp-needs-auth-cache.json  (PRIVATE)
  projects     -> ~/.claude-shared/projects
  todos        -> ~/.claude-shared/todos
  tasks        -> ~/.claude-shared/tasks
  plans        -> ~/.claude-shared/plans
  plugins      -> ~/.claude-shared/plugins
  settings.json -> ~/.claude-shared/settings.json
  history.jsonl -> ~/.claude-shared/history.jsonl
  CLAUDE.md    -> ~/.claude-shared/CLAUDE.md
  ... (every shared item is a symlink)

~/.claude-milos85vasic2nd/        <-- account 2 (claude2 alias)
  .credentials.json    (PRIVATE — claude2's OAuth tokens)
  .claude.json         (PRIVATE — claude2's onboarding state)
  mcp-needs-auth-cache.json  (PRIVATE)
  ... (same symlink set as above)
```

```
~/.claude/                          <-- user-scope default root
  CLAUDE.md    -> ~/.claude-shared/CLAUDE.md
  plugins/
    cache        -> ~/.claude-shared/plugins/cache
    marketplaces -> ~/.claude-shared/plugins/marketplaces
```

Properties of this layout:

1. **Account switching is purely a credentials swap.** Running `claude1` vs `claude2` changes `CLAUDE_CONFIG_DIR`, which changes which `.credentials.json` is loaded — everything else (history, memory, plugins) resolves through symlinks to the same shared store.
2. **Adding a new account is the same operation, parameterised.** A new `~/.claude-<name>` dir with the same symlink set + its own `.credentials.json` slots in seamlessly. See `claude-add-account.sh`.
3. **Account identity stays compartmentalised.** OAuth tokens are not in the shared store, so leaking the shared store does not leak account credentials.
4. **Rollback is mechanical.** Every original directory was moved to `<path>.preunify.<timestamp>` before the symlink replaced it. A single script (`claude-rollback.sh`) reverses the migration.

---

4. Reproducing this on any host

This section is the actual step-by-step procedure. It assumes you have two (or more) Claude Code accounts already set up the conventional way — one config directory per account, each with its own `.credentials.json`. If you don't have a second account yet, run the first three steps and then use `claude-add-account.sh` to create one.

4.1 Prerequisites

```
# Linux (Debian/Ubuntu/Fedora/Arch)
sudo apt-get install -y rsync jq gawk pandoc weasyprint    # Debian/Ubuntu
# or
sudo dnf install -y rsync jq gawk pandoc weasyprint        # Fedora
# or
sudo pacman -S rsync jq gawk pandoc weasyprint             # Arch

# macOS (Homebrew)
brew install rsync jq gawk pandoc weasyprint
```

`weasyprint` is optional but produces the best-looking PDF; the export script will fall back to `wkhtmltopdf` or `chromium --headless` if weasyprint isn't available.

4.2 Lay down the scripts directory

```
mkdir -p ~/Documents/scripts
# Copy or git-clone the contents of this repo's scripts/ into ~/
        Documents/scripts.
# At minimum you need:
#   lib.sh
#   install.sh
#   claude-unify.sh
#   claude-add-account.sh
#   claude-remove-account.sh
#   claude-list-accounts.sh
#   claude-rollback.sh
#   claude-export-docs.sh
chmod +x ~/Documents/scripts/*.sh
```

4.3 Run the installer

```
bash ~/Documents/scripts/install.sh
```

What `install.sh` does, in order:

1. Verifies `rsync`, `jq`, `awk` are on PATH; warns if `pandoc` is missing.
2. Symlinks every `claude-*.sh` script into `~/.local/bin/` so they're invokable as plain commands (`claude-unify`, `claude-add-account`, ...).
3. Adds `export PATH="$HOME/.local/bin:$PATH"` to `~/.bashrc` and `~/.zshrc` if missing.
4. Creates the managed alias file at `~/.local/share/claude-multi-account/aliases.sh` and ensures it's sourced from `~/.bashrc` and `~/.zshrc`.
5. Migrates any pre-existing inline `alias claude<N>=...` lines from `~/.bashrc` / `~/.zshrc` into the managed alias file, commenting out the originals (and saving a `.preunify.<timestamp>` backup of the rc file first).
6. Auto-detects every `~/.claude-*` directory and runs `claude-unify.sh` to merge them into `~/.claude-shared/`.
7. Re-renders the PDF and HTML from this markdown via `claude-export-docs.sh`.

After it finishes, reload your shell:

```
exec $SHELL -l
claude-list-accounts
```

You should see every account dir, with `CREDS: yes` and a `LINKS: 6/6` (or higher) indicator showing the shared symlinks are in place.

4.4 Add a third (or fourth, ...) account

```
claude-add-account
# prompts:
#   Alias name      [claude3]:
#   Config directory [/home/you/.claude-claude3]:
```

If you accept both defaults, you get:

- `~/.claude-claude3/` created with the full set of shared-state symlinks already in place.
- `alias claude3="CLAUDE_CONFIG_DIR=$HOME/.claude-claude3 \ $CLAUDE_BIN"` appended to the managed alias file.
- A reminder to authenticate: `claude3 /login`.

Custom alias names work just as well:

```
claude-add-account --alias work --dir ~/.claude-work --yes
work /login
```

4.5 Verify the result

```
claude-list-accounts

# Expected output (your alias names may differ):
# ALIAS         CONFIG DIR
        CREDS   LINKS  NOTES
# claude1       /home/you/.claude-milos85vasic
        yes     6/6
# claude2       /home/you/.claude-milos85vasic2nd
        yes     6/6
# claude3       /home/you/.claude-claude3
        no      6/6
#
# Shared store: /home/you/.claude-shared
# Alias file:   /home/you/.local/share/claude-multi-account/
        aliases.sh
```

You can confirm symlinks resolve correctly:

```
readlink -f ~/.claude-milos85vasic/projects
# /home/you/.claude-shared/projects

readlink -f ~/.claude-milos85vasic2nd/projects
# /home/you/.claude-shared/projects
```

```
# Both should print the same path. The same is true for todos,
        plans,
# settings.json, history.jsonl, CLAUDE.md, etc.
```

4.6 Rollback if needed

Every directory or file the unifier touched was moved to `<path>.preunify.<timestamp>` before the symlink was created. To restore:

```
claude-rollback
# moves ~/.claude-shared aside and restores every .preunify.*
        backup
```

The shared store isn't deleted — it's renamed to `~/.claude-shared.removed.<timestamp>` so you can manually recover any data written through the symlinks before rolling back.

---

5. Operational notes

5.1 Concurrent use of two accounts

`sessions/` is shared in this configuration. A live Claude Code session holds its lockfile under `sessions/`, and two concurrent processes writing to the same lockfile path can confuse each other. **Avoid running `claude1` and `claude2` simultaneously in two terminals.** If you need true concurrent use, override the unifier to keep `sessions/` private:

```
# in claude-unify.sh, remove "sessions" from SHARED_ITEMS
# then run claude-unify again — it'll relink sessions/ as per-
        account
```

`history.jsonl` is append-only and is mostly safe under concurrent writes (worst case: a small number of duplicate lines, which the next unify run will de-dupe).

5.2 Plugin path rewriting

Claude Code records absolute paths in `plugins/installed_plugins.json` and `plugins/known_marketplaces.json`. These paths can point to one of:

- `~/.claude/plugins/cache/...` (user-scope installs from any account)
- `~/.claude-<name>/plugins/cache/...` (project-scope installs from that account)

The unifier rsyncs plugin code from all three roots (`~/.claude/plugins/`, plus every account's `plugins/`) into `~/.claude-shared/plugins/`, then rewrites every `installPath` / `installLocation` to point under `~/.claude-shared/plugins/`. It also symlinks `~/.claude/plugins/cache` and `~/.claude/plugins/marketplaces` into the shared store, so any future user-scope install lands in the unified spot.

If you ever install a plugin and Claude reports "plugin code not found," re-run `claude-unify.sh` to repath the manifest.

5.3 Adding a new account from scratch (no existing dir)

The `claude-add-account.sh` script works whether or not the target dir exists yet. If it doesn't exist, the script creates an empty dir, links every shared item into it, and registers the alias. The new account has no credentials of its own — your next step is `claudeN /login`.

If you want to also pre-populate the new account with a `.claude.json` template (e.g., to skip the new-user warmup tips), copy one from an existing account *after* the add-account script finishes:

```
cp ~/.claude-milos85vasic2nd/.claude.json ~/.claude-
        claude3/.claude.json
# but DO NOT copy .credentials.json — that would leak the source
# account's OAuth tokens to the new account
```

5.4 Removing an account

```
claude-remove-account --alias claude3
# default: archives ~/.claude-claude3 -> ~/.claude-
        claude3.removed.<timestamp>
# add --delete to remove it instead
```

The shared store is left intact; other accounts continue to function.

5.5 Backups created during install

The installer never deletes existing data. Every directory or file it replaced lives at `<original-path>.preunify.<timestamp>` (or `<original-path>.removed.<timestamp>` for rollback). You can safely delete those backups once you've confirmed the new setup works:

```
find ~ -maxdepth 3 -name '*.preunify.*' -print     # see what's
        there
find ~ -maxdepth 3 -name '*.preunify.*' -delete    # delete once
        happy
```

A pre-migration tarball is also placed in `~/Documents/claude-config-backups/` by the original migration. Keep it until you're confident in the new layout.

---

6. Cross-platform reproduction

Linux

Everything in this document was developed and tested on Linux 6.12 with bash 5.2, jq 1.8.1, rsync 3.2.7, pandoc 3.9. No Linux-specific behaviours are relied on; the scripts pass `shellcheck` clean and use only POSIX-portable tools plus jq.

macOS

macOS ships an older bash (3.2) by default. Install GNU bash via Homebrew (`brew install bash`) and update the shebangs only if you hit syntax issues — the scripts use `bash 4+` features only where the fallback is explicit (`mapfile`, associative arrays). On a default macOS:

```
brew install bash jq rsync gawk pandoc weasyprint
echo "$(brew --prefix)/bin/bash" | sudo tee -a /etc/shells
chsh -s "$(brew --prefix)/bin/bash"
```

Everything else (paths, `$HOME`, `~/.claude-*` convention) is identical to Linux.

WSL / Windows Subsystem for Linux

WSL is just Linux for our purposes. Use the Linux instructions. If you also run Claude Code natively on Windows, the Windows version uses a different config root entirely (`%APPDATA%\claude`) and is not covered by this toolkit.

---

7. Tests

Everything in this toolkit is covered by a hermetic test suite under `~/Documents/scripts/tests/`. Each test runs against a temporary `$HOME` created by `mktemp -d -t cma-test.XXXX`; the real `~/.claude-*` directories are never touched. After the test process exits, the sandbox is `rm -rf`'d (with a safety check that the path basename starts with `cma-test.`).

7.1 Running the suite

```
# All tests:
bash ~/Documents/scripts/tests/run-all.sh

# A single file (no test_ prefix, no .sh suffix):
bash ~/Documents/scripts/tests/run-all.sh unify
bash ~/Documents/scripts/tests/run-all.sh add_remove
```

Output is line-based:

```
==> test_unify.sh

  • every shared item becomes a symlink into the shared store
    [PASS] .claude-acct1 projects linked: ...
    [PASS] .claude-acct2 projects linked: ...
  • settings.json enabledPlugins is the union {a,b,c}
    [PASS] union (.enabledPlugins | keys | sort | join(",") =
a,b,c)
    ...

✓ 33 passed, 0 failed
```

The orchestrator's exit code is 0 only if every `test_*.sh` file's own summary returned 0.

7.2 What each test covers

File	What it asserts
`test_lib.sh`	`cma_validate_alias` accepts/rejects correctly; `cma_suggest_alias` returns the next free `claudeN` (skipping past existing aliases, not just counting); `cma_write_alias` is idempotent; `cma_remove_alias` deletes; `cma_ensure_alias_file` patches `.bashrc`; `cma_detect_accounts` excludes the shared store.
`test_unify.sh`	Every shared item becomes a symlink to `$SHARED_DIR`; `settings.json` `enabledPlugins` is a strict union of every account's set; non-`enabledPlugins` top-level keys are preserved with right-most file winning; `history.jsonl` is concatenated and de-duplicated; on memory-file conflicts, the most recently touched account wins; `plugins/installed_plugins.json` and `known_marketplaces.json` have absolute paths rewritten to `$SHARED_DIR/plugins/`; `.credentials.json` and `.claude.json` are NOT symlinked; re-running `claude-unify.sh` produces a byte-identical `settings.json`

File	What it asserts
	(idempotency); N>2 accounts (adding acct3 and re-unifying) works; --rollback restores every .preunify.* backup and moves the shared store aside.
test_add_remove.sh	claude-add-account.sh --yes creates the dir with full symlink set and writes a shell-safe alias line; refuses to overwrite an existing dir; accepts a custom alias name + custom dir; validates alias names against the safe regex; claude-remove-account.sh --archive moves the dir aside and removes the alias; --delete actually deletes; unknown alias errors out.
test_list.sh	Empty host (no accounts) renders just the header; populated host renders one row per detected account dir; the shared store path is printed; accounts missing credentials show CREDS: no.

7.3 How the test harness works

Two small libraries sit under tests/lib/:

- assert.sh — it, assert_eq, assert_file, assert_dir, assert_symlink_to, assert_not_symlink, assert_file_contains, assert_file_not_contains, assert_jq, assert_lines, assert_exit. Each helper prints a [PASS] or [FAIL] line and adjusts the counters used by summary at end-of-file.
- sandbox.sh — make_sandbox (creates a tempdir and overrides HOME, SHARED_DIR, ALIAS_FILE, DEFAULT_DIR, CLAUDE_BIN), make_account NAME [--plugins] [--settings JSON] [--history a|b|c] [--memory KEY:VALUE] [--todo SUBJECT] (builds a realistic per-account fixture dir), plus thin wrappers run_unify, run_add_account, etc.

One subtlety worth knowing: lib.sh (the production helper) has set -euo pipefail. When a test sources it for unit-testing the helper functions, the set -e flag stays active in the test shell. The unit test file (test_lib.sh) therefore runs set +e immediately after sourcing lib.sh so that intentional failures (e.g. cma_validate_alias "bad name") don't abort the whole test process. End-to-end tests don't have this problem because they invoke the scripts as external processes (via run_unify, etc.), which run in their own shell.

7.4 Adding a new test

1. Create `tests/test_<feature>.sh`. Mark it executable.
2. Source the two libs:

```
TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"
source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"
```

3. Call `make_sandbox` once at the top.
4. Use `it "..."` to label each scenario, then a sequence of `assert_*` calls.
5. End the file with `summary` so the orchestrator picks up the result.
6. Re-run `bash tests/run-all.sh` — auto-discovery will pick it up.

---

8. The complete script source

For convenience and audit, the full source of the toolkit is reproduced below. The authoritative copies live at `~/Documents/scripts/`.

8.1 `lib.sh`

```
#!/usr/bin/env bash
# lib.sh — shared functions for the Claude multi-account toolkit.
# Sourced by the other scripts; no side effects on its own.

set -euo pipefail

# Which rc files to source the alias file from. macOS interactive
        shell is
# zsh; touching .bashrc there just creates noise the user has to
        clean up.
# On Linux we keep both because either may be the login shell.
if [[ "$(uname -s)" == "Darwin" ]]; then
  CMA_RC_FILES=("$HOME/.zshrc")
else
  CMA_RC_FILES=("$HOME/.bashrc" "$HOME/.zshrc")
fi

# Resolve paths the user can override. SHARED_DIR is the single
        canonical
# location for cross-account state; ALIAS_FILE is the rc-sourced
        file we
# manage aliases through; ACCOUNT_PREFIX is the dir-name prefix
        for new
# per-account config directories.
: "${SHARED_DIR:=$HOME/.claude-shared}"
: "${ALIAS_FILE:=$HOME/.local/share/claude-multi-account/
        aliases.sh}"
```

```
: "${ACCOUNT_PREFIX:=.claude-}"

CLAUDE_BIN_DEFAULT="${CLAUDE_BIN:-$HOME/.local/bin/claude}"

cma_log()  { printf '\033[36m[cma]\033[0m %s\n' "$*" >&2; }
cma_warn() { printf '\033[33m[cma warn]\033[0m %s\n' "$*" >&2; }
cma_err()  { printf '\033[31m[cma err]\033[0m %s\n' "$*" >&2; }
cma_die()  { cma_err "$*"; exit 1; }

cma_require() {
  command -v "$1" >/dev/null 2>&1 || cma_die "missing required
        tool: $1"
}

# Keys inside .claude.json that must NEVER leak across accounts
        (auth + identity).
# Everything else in .claude.json — projects map, UX state,
        caches, etc. — is
# safely shareable. New auth keys should be added here, not
        assumed shareable.
CMA_CLAUDE_JSON_PRIVATE_KEYS='["userID","oauthAccount","firstStartTime","claudeCod

# Deep-merge every account's .claude.json so each account's file
        ends up with
# its OWN auth keys + the UNION of all other top-level keys
        (rightmost-wins
# for scalar conflicts, recursive object merge for `projects` and
        friends).
#
# Args: one or more account config dirs.
# Side effect: rewrites each $acct/.claude.json in place. Skips an
        account
# that doesn't yet have a .claude.json (e.g. brand-new dir before
        first login).
cma_merge_claude_json() {
  local accts=("$@")
  (( ${#accts[@]} >= 1 )) || return 0
  cma_require jq

  # Build the merged "shared portion" (everything except private
        keys).
  local shared_tmp; shared_tmp="$(mktemp)"
  printf '{}\n' > "$shared_tmp"
  local acct prev="$shared_tmp"
  for acct in "${accts[@]}"; do
    local f="$acct/.claude.json"
    [[ -s "$f" ]] || continue
    local next; next="$(mktemp)"
    # $a (accumulator) * ($b stripped of private keys). jq's `*`
        is recursive
    # deep-merge for objects; rightmost wins on scalar conflicts.
    if ! jq -s --argjson priv "$CMA_CLAUDE_JSON_PRIVATE_KEYS" '
      . as [$a, $b]
```

```
      | ($b | with_entries(select(.key as $k | $priv | index($k) |
        not))) as $shared_b
      | ($a // {}) * ($shared_b // {})
    ' "$prev" "$f" > "$next"; then
      rm -f "$next"
      cma_warn ".claude.json in
        $acct is not valid JSON — skipping its contribution"
      continue
    fi
    rm -f "$prev"
    prev="$next"
  done

  # Now $prev holds the merged shared portion. Write each
        account's file with
  # its own private keys overlaid on top of the shared portion.
  for acct in "${accts[@]}"; do
    local f="$acct/.claude.json" out
    out="$(mktemp)"
    if [[ -s "$f" ]]; then
      # Guard against set -e: a corrupt $f makes jq fail, which
        would abort
      # the whole loop and leave the remaining accounts unsynced.
      if ! jq -s --argjson priv "$CMA_CLAUDE_JSON_PRIVATE_KEYS" '
        . as [$shared, $own]
        | ($own | with_entries(select(.key as $k | $priv |
        index($k)))) as $priv_only
        | ($shared // {}) * $priv_only
      ' "$prev" "$f" > "$out" 2>/dev/null; then
        rm -f "$out"
        cma_warn ".claude.json for $acct could not be merged
        (invalid JSON?) — leaving original alone"
        continue
      fi
    else
      # New account, no existing .claude.json. Seed with shared
        content only.
      cp "$prev" "$out"
    fi
    # Final sanity: only replace if the output is parseable JSON.
    if jq -e . "$out" >/dev/null 2>&1; then
      mv "$out" "$f"
    else
      rm -f "$out"
      cma_warn "merged .claude.json for $acct was invalid —
        leaving original file untouched"
    fi
  done
  rm -f "$prev"
}

# Detect Linux vs macOS for platform-specific commands.
cma_os() {
  case "$(uname -s)" in
```

```
    Linux*)   echo linux ;;
    Darwin*)  echo macos ;;
    *)        echo unknown ;;
  esac
}

# Find all existing Claude account config directories under $HOME,
        matching
# the convention `.claude-<name>`. Echoes absolute paths, one per
        line,
# sorted. The default `~/.claude` is intentionally excluded
        because we treat
# it as the shared user-scope spot, not an account dir.
cma_detect_accounts() {
  local d
  while IFS= read -r d; do
    [[ "$d" == *"-shared" ]] && continue
    # Provider-alias dirs (~/.claude-prov-<id>, created by claude-
        providers)
    # are account-like for SHARED state but must NEVER be merged
        into
    # real-account auth/identity or unify, so they're excluded
        from detection
    # exactly like *-shared. This is the linchpin that keeps the
        existing
    # claudeN accounts and add-account untouched by the provider
        feature.
    [[ "$(basename "$d")" == "${ACCOUNT_PREFIX}prov-"* ]] &&
        continue
    # Empty dirs always count (a brand-new account before any
        claude run).
    if [[ -z "$(ls -A "$d" 2>/dev/null)" ]]; then
      echo "$d"
      continue
    fi
    # Non-empty: must look like a Claude account (at least one
        tell-tale
    # file/dir). Filters out dirs that merely match the `.claude-
        *` prefix
    # but belong to other tools (e.g. `.claude-server-commander`).
    if [[ -d "$d/projects" || -d "$d/todos" || -d "$d/plugins" \
       || -f "$d/.claude.json" || -f "$d/.credentials.json" \
       || -f "$d/history.jsonl" ]]; then
      echo "$d"
    fi
  done < <(find "$HOME" -maxdepth 1 -type d -name "$
        {ACCOUNT_PREFIX}*" 2>/dev/null | sort)
}

# Read all aliases of the form `alias name=...` from $ALIAS_FILE
        and print
# their names (one per line). Returns nothing if the file is
        absent.
cma_existing_aliases() {
  [[ -f "$ALIAS_FILE" ]] || return 0
```

```
  awk -F'[ =]+' '/^[[:space:]]*alias[[:space:]]+/{print $2}'
        "$ALIAS_FILE"
}

# Suggest the next free `claude<N>` alias by scanning current
      aliases.
# E.g., if claude1 and claude2 exist, returns "claude3".
cma_suggest_alias() {
  local highest=0 n
  for a in $(cma_existing_aliases); do
    if [[ "$a" =~ ^claude([0-9]+)$ ]]; then
      n="${BASH_REMATCH[1]}"
      (( n > highest )) && highest="$n"
    fi
  done
  echo "claude$((highest + 1))"
}

# Validate that an alias name is safe to embed in a generated
      `alias ...=`
# line — strictly alphanumeric + underscore/hyphen, starting with
      a letter.
cma_validate_alias() {
  [[ "$1" =~ ^[a-zA-Z][a-zA-Z0-9_-]*$ ]] || cma_die
        "invalid alias name: $1"
}

# Ensure $ALIAS_FILE exists with the header sentinel and is
      sourced from
# the user's rc files. Idempotent — safe to call repeatedly.
cma_ensure_alias_file() {
  mkdir -p "$(dirname "$ALIAS_FILE")"
  if [[ ! -f "$ALIAS_FILE" ]]; then
    cat > "$ALIAS_FILE" <<EOF
# Managed by claude-multi-account. Do not edit by hand; use
# ~/.local/bin/claude-add-account to add accounts.
export CLAUDE_BIN="${CLAUDE_BIN_DEFAULT}"
EOF
  fi
  # Migration: rewrite any pre-wrapper alias lines that invoke
      $CLAUDE_BIN
  # directly so they go through cma_run instead. Pre-existing
      installs need
  # this on the first re-run of install.sh after the runtime-sync
      feature
  # landed; idempotent (a line already using cma_run is left
      alone).
  if grep -qE '^alias[[:space:]]+[^=]+=.*CLAUDE_CONFIG_DIR=[^ ]+
      [[:space:]]+\\?\$CLAUDE_BIN"$' "$ALIAS_FILE"; then
    local tmp; tmp="$(mktemp)"
    sed -E 's|(^alias[[:space:]]+[^=]+=)"(CLAUDE_CONFIG_DIR=[^ ]+)
      [[:space:]]+\\?\$CLAUDE_BIN"$|\1"\2 cma_run"|'
      "$ALIAS_FILE" > "$tmp"
    mv "$tmp" "$ALIAS_FILE"
```

```
    cma_log "migrated existing aliases in $ALIAS_FILE to use
        cma_run wrapper"
  fi
  # Ensure the cma_run wrapper is present in the alias file. This
        is the
  # runtime hook that keeps .claude.json projects/session state
        synchronized
  # across every account: pull merged state before launch, push
        back after exit.
  if ! grep -q '^cma_run\(\)' "$ALIAS_FILE"; then
    cat >> "$ALIAS_FILE" <<'EOF'

# Wrapper: keeps .claude.json projects/session index synced across
        every
# logged-in account. Pulls merged state from every account into
        the launching
# one before claude runs; pushes the post-session state back out
        after exit.
# Cheap (jq deep-merge of one ~50KB file per account), runs
        unconditionally.
cma_run() {
  if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" pull "$CLAUDE_CONFIG_DIR"
        2>/dev/null || true
  fi
  "$CLAUDE_BIN" "$@"
  local rc=$?
  if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" push "$CLAUDE_CONFIG_DIR"
        2>/dev/null || true
  fi
  return $rc
}
EOF
  fi
  # Ensure the cma_run_provider wrapper is present. This launches
        Claude Code
  # against a non-Anthropic provider: it reads the per-provider
        non-secret env
  # file, injects the API key from the keys file at launch (the
        toolkit never
  # persists secrets itself), then runs claude directly (native
        transport) or

        # via claude-code-router (router transport). Self-
        contained: the user's shell
  # sources only this alias file, not lib.sh.
  if ! grep -q '^cma_run_provider\(\)' "$ALIAS_FILE"; then
    cat >> "$ALIAS_FILE" <<'EOF'

cma_run_provider() {
  local id="$1"; shift 2>/dev/null || true
  local pdir="$HOME/.local/share/claude-multi-account/providers"
  local envf="$pdir/$id.env"
  if [[ ! -f "$envf" ]]; then
```

```
  printf 'claude-providers: unknown provider %s (missing %s)\n'
      "$id" "$envf" >&2
  return 1
fi
# shellcheck disable=SC1090
source "$envf"
local keysf="${CMA_KEYS_FILE:-$HOME/api_keys.sh}"
[[ -f "$keysf" ]] && { set -a; source "$keysf"; set +a; }
# Indirect-expand the key var name. ${!var} is bash-only and a
      fatal error in
# zsh (the default macOS interactive shell this alias file is
      sourced into),
# so use eval, which works in both. CMA_PROVIDER_KEYVAR is a
      validated env
# var name ([A-Za-z_][A-Za-z0-9_]*), so this eval is safe.
local token=""
eval "token=\"\${$CMA_PROVIDER_KEYVAR:-}\""
if [[ -z "$token" ]]; then
  printf 'claude-providers: $%s is empty (set it in %s)\n'
      "$CMA_PROVIDER_KEYVAR" "$keysf" >&2
  return 1
fi
export CLAUDE_CONFIG_DIR="$CMA_PROVIDER_CONFIG_DIR"
# Sync .claude.json projects/session index across ALL accounts
      and providers
# so sessions created under any alias are visible from every
      other alias.
# Pull merged state before launch; push post-session state after
      exit.
if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
  "$HOME/.local/bin/claude-sync-state" pull "$CLAUDE_CONFIG_DIR"
      2>/dev/null || true
fi
local rc
local _proxy_pid=""
if [[ "${CMA_PROVIDER_TRANSPORT:-native}" == "router" ]]; then
  if ! command -v ccr >/dev/null 2>&1; then
    printf 'claude-providers: provider %s needs claude-code-
      router.\n  Install: npm install -g @musistudio/claude-
      code-router\n' "$id" >&2
    return 127
  fi
  # Upsert THIS provider into ccr config with the live key
      (regenerated each
  # launch, chmod 600 — never stored by the toolkit), set it as
      the active
  # route, then launch through ccr.
  local cfg="$HOME/.claude-code-router/config.json"
      base="$CMA_PROVIDER_BASE_URL"
  # Create the dir + config with restrictive perms from the
      start: this file
  # will hold the live API key, so it must never be group/world
      readable,
  # even transiently or if a later jq rewrite fails.
  ( umask 077; mkdir -p "$HOME/.claude-code-router"
```

```
    [[ -f "$cfg" ]] || echo '{"Providers":[],"Router":{}}' >
      "$cfg" )
  chmod 600 "$cfg" 2>/dev/null || true
  case "$base" in
    */chat/completions|*/v1beta/models/|*/v1beta/models) ;;
    *) base="${base%/}/chat/completions" ;;
  esac
  # Start compatibility proxy if the provider needs one (e.g.
      Poe requires
  # `parameters` in tool definitions; Claude Code sometimes
      omits it).
  # Check for provider-specific proxy or base proxy (poe2 ->
      poe_proxy).
  local _base_id="${CMA_PROVIDER_ID%%[0-9]*}"
  local _proxy_script=""
  if [[ -x "$LIB_DIR/proxy/${CMA_PROVIDER_ID}_proxy.py" ]]; then
    _proxy_script="$LIB_DIR/proxy/${CMA_PROVIDER_ID}_proxy.py"
  elif [[ -x "$LIB_DIR/proxy/${_base_id}_proxy.py" ]]; then
    _proxy_script="$LIB_DIR/proxy/${_base_id}_proxy.py"
  fi
  if [[ -n "$_proxy_script" ]]; then
    local _proxy_port=3457
    python3 "$_proxy_script" --port "$_proxy_port" &
    _proxy_pid=$!
    local _waited=0
    while ! lsof -i :$_proxy_port >/dev/null 2>&1 && (( _waited
      < 25 )); do
      sleep 0.2
      _waited=$((_waited + 1))
    done
    base="http://127.0.0.1:${_proxy_port}/v1/chat/completions"
    cma_log "started proxy for $CMA_PROVIDER_ID on port
      $_proxy_port (pid=$_proxy_pid)"
  fi
  if command -v jq >/dev/null 2>&1; then
    local tmp; tmp="$(mktemp)"; chmod 600 "$tmp" 2>/dev/null ||
      true
    # Pass the secret through the environment ($ENV.tok), never
      as a jq argv
    # argument — argv is visible in ps/proc to other local
      users.
    if CMA_TOK="$token" jq --arg n "$CMA_PROVIDER_ID" --arg u
      "$base" \
          --arg s "$CMA_PROVIDER_MODEL" --arg f "$
      {CMA_PROVIDER_FAST_MODEL:-$CMA_PROVIDER_MODEL}" '
        .Providers = ([ .Providers[]? | select(.name != $n) ]
          + [{name:$n, api_base_url:$u, api_key:$ENV.CMA_TOK,
      models:[$s,$f],
              transformer:{use:
      ["cleancache","streamoptions"]}}])
        | .Router.default = ($n + "," + $s)
        | .Router.background = ($n + "," + $f)
      ' "$cfg" > "$tmp" 2>/dev/null; then
      mv "$tmp" "$cfg"; chmod 600 "$cfg" 2>/dev/null || true
```

```
        ccr restart >/dev/null 2>&1 || true
      else
        rm -f "$tmp"
      fi
    fi
    ccr code "$@"; rc=$?
    # Stop proxy if we started one
    if [[ -n "$_proxy_pid" ]]; then
      kill "$_proxy_pid" 2>/dev/null || true
      cma_log "stopped proxy for $CMA_PROVIDER_ID
        (pid=$_proxy_pid)"
    fi
  else
    export ANTHROPIC_BASE_URL="$CMA_PROVIDER_BASE_URL"
    export ANTHROPIC_AUTH_TOKEN="$token"
    export ANTHROPIC_MODEL="$CMA_PROVIDER_MODEL"
    [[ -n "${CMA_PROVIDER_FAST_MODEL:-}" ]] && export
        ANTHROPIC_SMALL_FAST_MODEL="$CMA_PROVIDER_FAST_MODEL"
    "$CLAUDE_BIN" "$@"; rc=$?
  fi
  # Push post-session state back to all accounts/providers for
        cross-alias visibility.
  if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" push "$CLAUDE_CONFIG_DIR"
        2>/dev/null || true
  fi
  return $rc
}
EOF
  fi
  local rc src_line="source \"$ALIAS_FILE\""
  for rc in "${CMA_RC_FILES[@]}"; do
    [[ -f "$rc" ]] || continue
    if ! grep -F -q "$src_line" "$rc"; then
      printf '\n# Claude multi-account aliases\n%s\n' "$src_line"
        >> "$rc"
      cma_log "added source line to $rc"
    fi
  done
}

# Add (or refresh) a single alias entry in $ALIAS_FILE.
        Idempotent.
# The generated alias wraps `$CLAUDE_BIN` with sync-pull (before
        launch) and
# sync-push (after exit) so the project/session index
        inside .claude.json
# stays merged across every logged-in account without manual unify
        runs.
cma_write_alias() {
  local alias_name="$1" config_dir="$2"
  cma_validate_alias "$alias_name"
  cma_ensure_alias_file
  # Strip any prior line for this alias, then append the new one.
```

```
  local tmp; tmp="$(mktemp)"
  grep -v -E "^alias[[:space:]]+${alias_name}=" "$ALIAS_FILE" >
        "$tmp" || true
  # Note: bash aliases can't take args, so we use a quoted
        CLAUDE_CONFIG_DIR= prefix
  # plus a wrapped invocation. The wrapper is a shell function
        reference (cma_run)
  # defined alongside in the alias file (added once by
        cma_ensure_alias_file).
  printf 'alias %s="CLAUDE_CONFIG_DIR=%s cma_run"\n' \
    "$alias_name" "$config_dir" >> "$tmp"
  mv "$tmp" "$ALIAS_FILE"
}

# Remove an alias line. Idempotent.
cma_remove_alias() {
  local alias_name="$1"
  [[ -f "$ALIAS_FILE" ]] || return 0
  local tmp; tmp="$(mktemp)"
  grep -v -E "^alias[[:space:]]+${alias_name}=" "$ALIAS_FILE" >
        "$tmp" || true
  mv "$tmp" "$ALIAS_FILE"
}

# ============================================================================
# Provider-alias helpers (used by claude-providers.sh)
# ============================================================================

# The shared items every account/provider dir symlinks into
        $SHARED_DIR.
# Kept here (single source) so claude-add-account and claude-
        providers agree.
CMA_SHARED_ITEMS=(
  projects todos tasks plans file-history paste-cache shell-
        snapshots
  session-env telemetry sessions backups cache plugins
  stats-cache.json history.jsonl settings.json CLAUDE.md
)

cma_providers_dir() { echo "$HOME/.local/share/claude-multi-
        account/providers"; }

# Symlink every shared item into a config dir (account or
        provider), creating
# empty placeholders in $SHARED_DIR for any item that doesn't
        exist yet.
# Idempotent: skips items already present in the target.
cma_link_shared_items() {
  local cdir="$1" item src tgt
  mkdir -p "$SHARED_DIR" "$cdir"
  for item in "${CMA_SHARED_ITEMS[@]}"; do
    src="$SHARED_DIR/$item"; tgt="$cdir/$item"
    if [[ ! -e "$src" ]]; then
      case "$item" in
```

```
        *.json|*.jsonl|*.md) : > "$src" ;;
        *) mkdir -p "$src" ;;
      esac
    fi
    [[ -e "$tgt" || -L "$tgt" ]] || ln -s "$src" "$tgt"
  done
}

# Force-enable the always-on plugins in the shared settings.json
        enabledPlugins
# map (additive union — never removes a user's existing entries).
        Each arg is a
# plugin key as it appears in enabledPlugins (e.g.
        "superpowers@anthropics").
cma_enable_plugins() {
  cma_require jq
  local settings="$SHARED_DIR/settings.json" tmp
  mkdir -p "$SHARED_DIR"
  [[ -s "$settings" ]] || printf '{}\n' > "$settings"
  tmp="$(mktemp)"
  local args=() p
  for p in "$@"; do args+=(--arg "p$((${#args[@]}/2))" "$p"); done
  # Build a jq program that sets each provided key true if absent.
  local prog='.enabledPlugins //= {}'
  local i=0
  for p in "$@"; do
    prog+=" | .enabledPlugins[\$p$i] //= true"; i=$((i+1))
  done
  if jq "${args[@]}" "$prog" "$settings" > "$tmp" 2>/dev/null;
        then
    mv "$tmp" "$settings"
  else
    rm -f "$tmp"; cma_warn "could not update enabledPlugins in
        $settings"
  fi
}

# Write the non-secret per-provider env file consumed by
        cma_run_provider.
# Args: id keyvar transport base_url model fast_model config_dir
cma_provider_write_env() {
  local id="$1" keyvar="$2" transport="$3" base="$4" model="$5"
        fast="$6" cdir="$7"
  # Normalize the literal "null" (from a missing JSON field) to
        empty so it
  # never leaks into the wrapper as a bogus value.
  [[ "$base" == "null" ]] && base=""
  [[ "$fast" == "null" ]] && fast=""
  local pdir; pdir="$(cma_providers_dir)"; mkdir -p "$pdir"
  # Values are single-quoted (with embedded-quote escaping) so
        sourcing the file
  # in the user's shell is safe regardless of characters in URLs/
        model ids.
  _cma_q() { printf "'%s'" "${1//\'/\'\\\'\'}"; }
```

```
  cat > "$pdir/$id.env" <<EOF
# generated by claude-providers — non-secret. Do not edit by hand.
# Secrets are NEVER stored here; the key is read from the keys
        file at launch.
CMA_PROVIDER_ID=$(_cma_q "$id")
CMA_PROVIDER_KEYVAR=$(_cma_q "$keyvar")
CMA_PROVIDER_TRANSPORT=$(_cma_q "$transport")
CMA_PROVIDER_BASE_URL=$(_cma_q "$base")
CMA_PROVIDER_MODEL=$(_cma_q "$model")
CMA_PROVIDER_FAST_MODEL=$(_cma_q "$fast")
CMA_PROVIDER_CONFIG_DIR=$(_cma_q "$cdir")
EOF
  unset -f _cma_q
}

# Write (or refresh) a provider alias: alias
        <name>="cma_run_provider <id>".
cma_provider_write_alias() {
  local alias_name="$1" id="$2"
  cma_validate_alias "$alias_name"
  cma_ensure_alias_file
  local tmp; tmp="$(mktemp)"
  grep -v -E "^alias[[:space:]]+${alias_name}=" "$ALIAS_FILE" >
        "$tmp" || true
  printf 'alias %s="cma_run_provider %s"\n' "$alias_name" "$id"
        >> "$tmp"
  mv "$tmp" "$ALIAS_FILE"
}
```

8.2 `claude-unify.sh`

```
#!/usr/bin/env bash
# claude-unify.sh — Unify N Claude Code account config dirs into a
        single
# shared store so every account sees the same projects/
        conversations,
# memory, history, todos, plans, plugins, and settings.
#
# Inputs:
#   * Positional args: one or more account config directories. If
        none
#     are given, all `~/.claude-*` directories are auto-detected
        (excluding
#     the shared store itself).
#   * Env vars:
#       SHARED_DIR    target shared store (default: ~/.claude-
        shared)
#       DEFAULT_DIR   user-scope plugin root (default: ~/.claude)
#
# Use --rollback to restore the .preunify.* backups and remove the
        shared
# store. Safe to re-run any time.
```

```
set -euo pipefail

# macOS ships bash 3.2 which lacks `mapfile`. Re-exec under
        Homebrew bash
# if available; otherwise tell the user how to install it.
if (( BASH_VERSINFO[0] < 4 )); then
  for newer in /opt/homebrew/bin/bash /usr/local/bin/bash; do
    [[ -x "$newer" ]] && exec "$newer" "$0" "$@"
  done
  echo "claude-unify requires bash 4+. Install via: brew install
        bash" >&2
  exit 1
fi

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
        ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src="$_cma_tgt" ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

DEFAULT_DIR="${DEFAULT_DIR:-$HOME/.claude}"

cma_require rsync
cma_require jq
cma_require awk

# Items each account dir contributes to the shared store. Order
        matters
# only insofar as we want plugins last so the path-rewrite step
        has all
# manifest entries to rewrite at once.
SHARED_ITEMS=(
  projects
  todos
  tasks
  plans
  file-history
  paste-cache
  shell-snapshots
  session-env
  telemetry
  sessions
  backups
  cache
  stats-cache.json
  history.jsonl
  settings.json
```

```
  plugins
)

PRIVATE_ITEMS=(
  .credentials.json
  .claude.json
  mcp-needs-auth-cache.json
)

ts() { date +%Y%m%d%H%M%S; }

already_linked_to_shared() {
  [[ -L "$1" ]] || return 1
  [[ "$(readlink -f "$1")" == "$(readlink -f "$2" 2>/dev/null ||
        echo "$2")" ]]
}

backup_and_remove() {
  local p="$1"
  [[ -e "$p" || -L "$p" ]] || return 0
  mv "$p" "${p}.preunify.$(ts)"
}

link_to_shared() {
  local item="$1" acct
  for acct in "${ACCOUNTS[@]}"; do
    [[ -d "$acct" ]] || continue
    local target="$acct/$item"
    already_linked_to_shared "$target" "$SHARED_DIR/$item" &&
        continue
    backup_and_remove "$target"
    mkdir -p "$(dirname "$target")"
    ln -s "$SHARED_DIR/$item" "$target"
  done
}

merge_dir_into_shared() {
  local item="$1" acct rc
  mkdir -p "$SHARED_DIR/$item"
  # First pass: each account fills only gaps (--ignore-existing)
        so the
  # union is preserved across N accounts.
  for acct in "${ACCOUNTS[@]}"; do
    if [[ -d "$acct/$item" && ! -L "$acct/$item" ]]; then
      rc=0
      rsync -a --ignore-existing "$acct/$item/" "$SHARED_DIR/
        $item/" || rc=$?
      # rsync exit 23/24 are partial transfers ("some files
        vanished" / "some
      # files couldn't be transferred") that we treat as warnings
        — common on
      # macOS for `unlinkat: Directory not empty` when symlinks
        straddle the
      # tree. Anything else is fatal.
```

```
      (( rc == 0 || rc == 23 || rc == 24 )) || return $rc
    fi
  done
  # Second pass: the LAST account (assumed most recently active)
        overlays
  # its versions for files that exist in multiple accounts. This
        biases
  # towards the freshest source for conflicting files like memory/
        *.md.
  local last="${ACCOUNTS[-1]}"
  if [[ -d "$last/$item" && ! -L "$last/$item" ]]; then
    rc=0
    rsync -a "$last/$item/" "$SHARED_DIR/$item/" || rc=$?
    (( rc == 0 || rc == 23 || rc == 24 )) || return $rc
  fi
}

merge_history_jsonl() {
  local acct tmp; tmp="$(mktemp)"
  for acct in "${ACCOUNTS[@]}"; do
    local f="$acct/history.jsonl"
    if [[ -f "$f" && ! -L "$f" ]]; then cat "$f" >> "$tmp"; fi
  done
  if [[ -f "$SHARED_DIR/history.jsonl" && ! -L "$SHARED_DIR/
        history.jsonl" ]]; then
    cat "$SHARED_DIR/history.jsonl" >> "$tmp"
  fi
  if [[ -s "$tmp" ]]; then
    awk 'NF && !seen[$0]++' "$tmp" > "$SHARED_DIR/history.jsonl"
  else
    : > "$SHARED_DIR/history.jsonl"
  fi
  rm -f "$tmp"
  return 0
}

merge_settings_json() {
  local acct files=() resolved
  for acct in "${ACCOUNTS[@]}"; do
    local f="$acct/settings.json"
    if [[ -L "$f" ]]; then
      resolved="$(readlink -f "$f")"
      if [[ -f "$resolved" ]]; then files+=("$resolved"); fi
    elif [[ -f "$f" ]]; then
      files+=("$f")
    fi
  done
  # De-duplicate. On re-runs both account symlinks resolve to the
        same
  # shared file, and passing the same path twice plus redirecting
        back to
  # it truncates the file before jq reads it.
  if (( ${#files[@]} > 1 )); then
```

```
    mapfile -t files < <(printf '%s\n' "${files[@]}" | awk '!
        seen[$0]++')
  fi
  case "${#files[@]}" in
    0) return 0 ;;
    1) cp -p "${files[0]}" "$SHARED_DIR/settings.json.tmp"
       mv "$SHARED_DIR/settings.json.tmp" "$SHARED_DIR/
        settings.json" ;;
    *)
      # Right-most file's top-level keys win, except
        enabledPlugins which
      # is the union across all accounts (any "true" survives).
      # We bind the slurped array to $all so the inner reductions
        don't
      # iterate over the values of the partially-merged outer
        object.
      # Write to a temp file first so we never truncate-then-read
        shared.
      jq -s '
        . as $all
        | (reduce $all[] as $x ({}; . * $x))
        | .enabledPlugins = (reduce $all[] as $x ({}; . +
        ($x.enabledPlugins // {})))
      ' "${files[@]}" > "$SHARED_DIR/settings.json.tmp"
      mv "$SHARED_DIR/settings.json.tmp" "$SHARED_DIR/
        settings.json"
      ;;
  esac
  return 0
}

merge_file_into_shared() {
  local item="$1" acct newest=""
  for acct in "${ACCOUNTS[@]}"; do
    local f="$acct/$item"
    if [[ -L "$f" ]]; then continue; fi
    if [[ ! -f "$f" ]]; then continue; fi
    if [[ -z "$newest" || "$f" -nt "$newest" ]]; then
        newest="$f"; fi
  done
  if [[ -n "$newest" ]]; then cp -p "$newest" "$SHARED_DIR/
        $item"; fi
  return 0
}

absorb_default_plugins() {
  [[ -d "$DEFAULT_DIR/plugins" && ! -L "$DEFAULT_DIR/plugins" ]]
        || return 0
  mkdir -p "$SHARED_DIR/plugins"
  local rc=0
  rsync -a --ignore-existing "$DEFAULT_DIR/plugins/" "$SHARED_DIR/
        plugins/" || rc=$?
  (( rc == 0 || rc == 23 || rc == 24 )) || return $rc
  return 0
```

```
}

rewrite_plugin_paths() {
  local f="$SHARED_DIR/plugins/installed_plugins.json"
  if [[ -f "$f" ]]; then
    local tmp; tmp="$(mktemp)"
    jq --arg shared "$SHARED_DIR" '
      .plugins |= with_entries(
        .value |= map(
          if ((.installPath // "") | test(".*/plugins/cache/"))
          then .installPath = ($shared + "/plugins/cache/" +
        (.installPath | sub(".*/plugins/cache/"; "")))
          else . end
        )
      )' "$f" > "$tmp" && mv "$tmp" "$f"
  fi
  local km="$SHARED_DIR/plugins/known_marketplaces.json"
  if [[ -f "$km" ]]; then
    local tmp; tmp="$(mktemp)"
    jq --arg shared "$SHARED_DIR" '
      with_entries(
        if ((.value.installLocation // "") | test(".*/plugins/
        marketplaces/"))
        then .value.installLocation = ($shared + "/plugins/
        marketplaces/" + (.value.installLocation | sub(".*/
        plugins/marketplaces/"; "")))
        else . end
      )' "$km" > "$tmp" && mv "$tmp" "$km"
  fi
}

# Repoint ~/.claude/plugins/cache and ~/.claude/plugins/
        marketplaces at
# the shared store so future user-scope installs land in one
        place.
link_default_plugin_subdirs() {
  for sub in cache marketplaces; do
    local src="$SHARED_DIR/plugins/$sub" tgt="$DEFAULT_DIR/
        plugins/$sub"
    [[ -d "$src" ]] || continue
    already_linked_to_shared "$tgt" "$src" && continue
    mkdir -p "$DEFAULT_DIR/plugins"
    backup_and_remove "$tgt"
    ln -s "$src" "$tgt"
  done
}

# CLAUDE.md (user-scope memory) lives at ~/.claude/CLAUDE.md by
        default.
# We promote it to the shared store and symlink all three
        locations (each
# account dir + ~/.claude) so any account writing user memory
        updates the
# same file.
```

```
sync_claude_md() {
  local shared_md="$SHARED_DIR/CLAUDE.md" src
  if [[ ! -f "$shared_md" ]]; then
    if [[ -f "$DEFAULT_DIR/CLAUDE.md" && ! -L "$DEFAULT_DIR/
        CLAUDE.md" ]]; then
      cp -p "$DEFAULT_DIR/CLAUDE.md" "$shared_md"
    else
      for acct in "${ACCOUNTS[@]}"; do
        if [[ -f "$acct/CLAUDE.md" && ! -L "$acct/CLAUDE.md" ]];
        then
          cp -p "$acct/CLAUDE.md" "$shared_md"; break
        fi
      done
    fi
    [[ -f "$shared_md" ]] || return 0
  fi
  local targets=("$DEFAULT_DIR/CLAUDE.md")
  for acct in "${ACCOUNTS[@]}"; do targets+=("$acct/CLAUDE.md");
        done
  for tgt in "${targets[@]}"; do
    [[ -d "$(dirname "$tgt")" ]] || continue
    already_linked_to_shared "$tgt" "$shared_md" && continue
    backup_and_remove "$tgt"
    ln -s "$shared_md" "$tgt"
  done
}

rollback() {
  cma_log "rollback: restoring .preunify.* backups"
  local root
  local roots=("$DEFAULT_DIR" "$DEFAULT_DIR/plugins"
        "$SHARED_DIR")
  for root in "${ACCOUNTS[@]}"; do roots+=("$root"); done
  for root in "${roots[@]}"; do
    [[ -d "$root" ]] || continue
    while IFS= read -r -d '' bk; do
      local orig="${bk%.preunify.*}"
      [[ -L "$orig" ]] && rm -f "$orig"
      [[ -e "$orig" ]] || { mv "$bk" "$orig"; cma_log "restored
        $orig"; }
    done < <(find "$root" -maxdepth 1 -name '*.preunify.*'
        -print0 2>/dev/null)
  done
  if [[ -d "$SHARED_DIR" ]]; then
    mv "$SHARED_DIR" "${SHARED_DIR}.removed.$(ts)"
    cma_log "moved $SHARED_DIR aside"
  fi
}

usage() {
  cat <<EOF
Usage: $(basename "$0") [--rollback] [account-dir ...]
```

```
Without args, auto-detects all ~/${ACCOUNT_PREFIX}* directories.
With --rollback, restores .preunify.* backups and removes the
        shared store.

Env: SHARED_DIR=$SHARED_DIR DEFAULT_DIR=$DEFAULT_DIR
EOF
}

# === main ===

ACCOUNTS=()
DO_ROLLBACK=0
while (( $# )); do
  case "$1" in
    -h|--help) usage; exit 0 ;;
    --rollback) DO_ROLLBACK=1; shift ;;
    --) shift; while (( $# )); do ACCOUNTS+=("$1"); shift; done ;;
    -*) cma_die "unknown flag: $1" ;;
    *)  ACCOUNTS+=("$1"); shift ;;
  esac
done

if (( ${#ACCOUNTS[@]} == 0 )); then
  mapfile -t ACCOUNTS < <(cma_detect_accounts)
fi

(( ${#ACCOUNTS[@]} >= 1 )) || cma_die "no account dirs given and
        none auto-detected at ~/${ACCOUNT_PREFIX}*"

cma_log "accounts: ${ACCOUNTS[*]}"
cma_log "shared:   $SHARED_DIR"

if (( DO_ROLLBACK )); then rollback; exit 0; fi

mkdir -p "$SHARED_DIR"
absorb_default_plugins

for item in "${SHARED_ITEMS[@]}"; do
  case "$item" in
    history.jsonl)    merge_history_jsonl ;;
    settings.json)    merge_settings_json ;;
    stats-cache.json) merge_file_into_shared "$item" ;;
    *)                merge_dir_into_shared "$item" ;;
  esac
  [[ "$item" == "plugins" ]] && rewrite_plugin_paths
  link_to_shared "$item"
  cma_log "ok: $item"
done

link_default_plugin_subdirs
sync_claude_md
```

```
# Merge the projects/session index inside .claude.json across
        every account.
# This is the single most important step for cross-account session
        resume:
# without it, account A's session UUIDs are invisible to account B
        even though
# the JSONL transcripts are already shared on disk via projects/.
cma_merge_claude_json "${ACCOUNTS[@]}"
cma_log "ok: .claude.json (projects/session index merged; auth
        keys preserved per-account)"

cma_log "done. shared: $SHARED_DIR"
cma_log "private per-account: ${PRIVATE_ITEMS[*]}"
```

8.3 `claude-add-account.sh`

```
#!/usr/bin/env bash
# claude-add-account.sh — Add a new Claude Code account to the
        multi-account
# setup. Creates the per-account config directory, links every
        shared item
# to the shared store, and registers a shell alias so the new
        account is
# invocable as e.g. `claude3`.
#
# Modes:
#   * Interactive (default): prompts for alias name and config dir
        name,
#     showing defaults that follow the existing pattern.
#   * Non-interactive: pass --alias NAME and optionally --dir
        PATH, e.g.
#     `claude-add-account.sh --alias work --dir ~/.claude-work`.
#
# After this script runs you still need to authenticate the new
        account
# with Anthropic — the script prints the exact command for that.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
        ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src="$_cma_tgt" ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

ALIAS_NAME=""
CONFIG_DIR=""
```

```
NONINTERACTIVE=0

usage() {
  cat <<EOF
Usage: $(basename "$0") [--alias NAME] [--dir PATH] [--yes]

  --alias NAME   Shell alias to create (e.g. claude3 or work).
        Default:
                 next free claudeN.
  --dir   PATH   Config directory for the new account. Default:
                 ~/${ACCOUNT_PREFIX}<alias>.
  --yes          Skip prompts and use defaults / passed values.
EOF
}

while (( $# )); do
  case "$1" in
    -h|--help) usage; exit 0 ;;
    --alias)   ALIAS_NAME="$2"; shift 2 ;;
    --dir)     CONFIG_DIR="$2"; shift 2 ;;
    --yes|-y)  NONINTERACTIVE=1; shift ;;
    *)         cma_die "unknown arg: $1" ;;
  esac
done

# Prompt with a default value, return whatever the user types (or
        default).
prompt() {
  local label="$1" default="$2" answer
  if (( NONINTERACTIVE )); then echo "$default"; return; fi
  read -r -p "$label [$default]: " answer < /dev/tty
  echo "${answer:-$default}"
}

[[ -n "$ALIAS_NAME" ]] || ALIAS_NAME="$(cma_suggest_alias)"
ALIAS_NAME="$(prompt "Alias name" "$ALIAS_NAME")"
cma_validate_alias "$ALIAS_NAME"

# Reject if alias already exists in the alias file.
if cma_existing_aliases | grep -qx "$ALIAS_NAME"; then
  cma_die "alias '$ALIAS_NAME' already exists in $ALIAS_FILE"
fi

[[ -n "$CONFIG_DIR" ]] || CONFIG_DIR="$HOME/${ACCOUNT_PREFIX}$
        {ALIAS_NAME}"
CONFIG_DIR="$(prompt "Config directory" "$CONFIG_DIR")"
case "$CONFIG_DIR" in /*) ;; *) CONFIG_DIR="$HOME/$CONFIG_DIR" ;;
        esac

[[ -d "$CONFIG_DIR" ]] && cma_die "config dir already exists:
        $CONFIG_DIR (refusing to overwrite)"

mkdir -p "$CONFIG_DIR"
```

```
cma_log "created $CONFIG_DIR"

# Symlink every shared item into the new account dir, using the
        single
# canonical list (CMA_SHARED_ITEMS in lib.sh) so accounts and
        provider
# aliases stay in lockstep without two copies of the list drifting
        apart.
cma_link_shared_items "$CONFIG_DIR"
cma_log "linked ${#CMA_SHARED_ITEMS[@]} shared items into
        $CONFIG_DIR"

# Add the alias. lib.sh handles dedupe, rc-file sourcing, etc.
cma_write_alias "$ALIAS_NAME" "$CONFIG_DIR"
cma_log "registered alias: $ALIAS_NAME -> $CONFIG_DIR"

cat <<EOF

[done] new account: $ALIAS_NAME

Next steps:
  1. Reload your shell (or run: source $ALIAS_FILE).
  2. Authenticate the new account:
       $ALIAS_NAME /login
  3. After login, run any project — history, memory, plugins, and
     settings will already match your other accounts via
        $SHARED_DIR.

To remove this account later:
  $LIB_DIR/claude-remove-account.sh --alias $ALIAS_NAME
EOF
```

8.4 claude-remove-account.sh

```
#!/usr/bin/env bash
# claude-remove-account.sh — Remove a Claude Code account from the
# multi-account setup. Drops the alias from the managed alias file
        and
# (optionally) deletes or archives the per-account config
        directory.
#
# The shared store is left untouched — only this one account's
        symlinks
# and credentials are removed. Other accounts continue using
        shared.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
        ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
```

```
  case "$_cma_tgt" in /*) _cma_src="$_cma_tgt" ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

ALIAS_NAME=""
DELETE_DIR=0
ARCHIVE_DIR=1
NONINTERACTIVE=0

usage() {
  cat <<EOF
Usage: $(basename "$0") --alias NAME [--delete | --archive] [--
        yes]

  --alias NAME   Required. Alias to remove (e.g. claude3).
  --delete       Permanently delete the per-account config
        directory.
  --archive      Move the per-account dir to
        <dir>.removed.<timestamp>
                 instead of deleting (default).
  --yes          Skip confirmation prompts.
EOF
}

while (( $# )); do
  case "$1" in
    -h|--help) usage; exit 0 ;;
    --alias)   ALIAS_NAME="$2"; shift 2 ;;
    --delete)  DELETE_DIR=1; ARCHIVE_DIR=0; shift ;;
    --archive) ARCHIVE_DIR=1; DELETE_DIR=0; shift ;;
    --yes|-y)  NONINTERACTIVE=1; shift ;;
    *)         cma_die "unknown arg: $1" ;;
  esac
done

[[ -n "$ALIAS_NAME" ]] || { usage; exit 2; }
cma_validate_alias "$ALIAS_NAME"

# Resolve the config dir by inspecting the existing alias line.
CONFIG_DIR=""
if [[ -f "$ALIAS_FILE" ]]; then
  # POSIX/2-arg awk match() only: the 3-arg capture form is GNU-
        awk-specific

        # and silently yields an empty CONFIG_DIR on the BSD awk
        shipped with macOS.
  CONFIG_DIR="$(awk -v a="$ALIAS_NAME" '
    $0 ~ "^alias[[:space:]]+"a"=" {
      if (match($0, /CLAUDE_CONFIG_DIR=[^ ]+/)) {
        s = substr($0, RSTART, RLENGTH)
```

```
        sub(/^CLAUDE_CONFIG_DIR=/, "", s)
        print s
      }
      exit
    }' "$ALIAS_FILE")"
fi
[[ -n "$CONFIG_DIR" ]] || cma_die "alias '$ALIAS_NAME' not found
        in $ALIAS_FILE"

cma_log "removing alias '$ALIAS_NAME' (config dir: $CONFIG_DIR)"

if (( ! NONINTERACTIVE )); then
  read -r -p "Proceed? [y/N] " ans < /dev/tty
  [[ "$ans" =~ ^[Yy]$ ]] || cma_die "aborted"
fi

cma_remove_alias "$ALIAS_NAME"
cma_log "alias removed from $ALIAS_FILE"

if [[ -d "$CONFIG_DIR" ]]; then
  if (( DELETE_DIR )); then
    rm -rf -- "$CONFIG_DIR"
    cma_log "deleted $CONFIG_DIR"
  else
    mv -- "$CONFIG_DIR" "${CONFIG_DIR}.removed.$(date +
        %Y%m%d%H%M%S)"
    cma_log "archived $CONFIG_DIR -> ${CONFIG_DIR}.removed.*"
  fi
fi

cat <<EOF

[done] account '$ALIAS_NAME' removed.

Notes:
  * Shared store $SHARED_DIR is untouched — other accounts still
        work.
  * Reload your shell so the alias change takes effect.
EOF
```

8.5 claude-list-accounts.sh

```
#!/usr/bin/env bash
# claude-list-accounts.sh — Print a status summary of every
        detected
# Claude Code account: which alias maps to it, whether credentials
        are
# present, and whether its shared-state symlinks are intact.

set -euo pipefail

if (( BASH_VERSINFO[0] < 4 )); then
  for newer in /opt/homebrew/bin/bash /usr/local/bin/bash; do
```

```
    [[ -x "$newer" ]] && exec "$newer" "$0" "$@"
  done
  echo "claude-list-accounts requires bash 4+. Install via: brew
        install bash" >&2
  exit 1
fi

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
        ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src="$_cma_tgt" ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

# Build alias -> dir mapping from the managed alias file.
declare -A ALIAS_OF_DIR
if [[ -f "$ALIAS_FILE" ]]; then
  while IFS= read -r line; do
    [[ "$line" =~ ^alias[[:space:]]+([a-zA-Z0-9_-]
        +)=.*CLAUDE_CONFIG_DIR=([^[:space:]]+) ]] || continue
    ALIAS_OF_DIR["${BASH_REMATCH[2]}"]="${BASH_REMATCH[1]}"
  done < "$ALIAS_FILE"
fi

# Items we expect to be symlinks pointing into $SHARED_DIR.
CHECK_LINKS=(projects history.jsonl settings.json plugins todos
        CLAUDE.md)

printf '%-12s  %-45s  %-6s  %-5s  %s\n' \
  "ALIAS" "CONFIG DIR" "CREDS" "LINKS" "NOTES"

mapfile -t accounts < <(cma_detect_accounts)
for dir in "${accounts[@]}"; do
  alias_name="${ALIAS_OF_DIR[$dir]:--}"
  if [[ -f "$dir/.credentials.json" ]]; then creds="yes"; else
        creds="no"; fi
  ok=0 total=0 missing=()
  for item in "${CHECK_LINKS[@]}"; do
    total=$((total+1))
    if [[ -L "$dir/$item" ]] \
       && [[ "$(readlink -f "$dir/$item")" == "$(readlink -f
         "$SHARED_DIR/$item" 2>/dev/null || echo "$SHARED_DIR/
         $item")" ]]; then
      ok=$((ok+1))
    else
      missing+=("$item")
    fi
  done
```

```
  notes=""
  (( ${#missing[@]} )) && notes="not linked: ${missing[*]}"
  printf '%-12s  %-45s  %-6s  %-5s  %s\n' \
    "$alias_name" "$dir" "$creds" "$ok/$total" "$notes"
done

echo
echo "Shared store: $SHARED_DIR"
[[ -d "$SHARED_DIR" ]] || echo "  (does not exist — run claude-
        unify.sh)"
echo "Alias file:   $ALIAS_FILE"
[[ -f "$ALIAS_FILE" ]] || echo "  (does not exist — run
        install.sh)"
```

8.6 claude-rollback.sh

```
#!/usr/bin/env bash
# claude-rollback.sh — Convenience wrapper that calls claude-
        unify.sh with
# --rollback. Restores every .preunify.<timestamp> backup created
        by the
# unification run and archives the shared store out of the way.
#
# After this you can re-run install.sh to start over from scratch.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
        ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src="$_cma_tgt" ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
exec "$LIB_DIR/claude-unify.sh" --rollback "$@"
```

8.7 claude-export-docs.sh

```
#!/usr/bin/env bash
# claude-export-docs.sh — Convert the multi-account markdown doc
        to
# matching .html and .pdf siblings. Re-runnable; overwrites
        outputs.
#
# Order of preference for PDF: pandoc + weasyprint > pandoc +
        wkhtmltopdf
# > chromium --headless print-to-pdf > weasyprint on the generated
        HTML.
# HTML always uses pandoc with a self-contained option so the file
        is
```

```
# portable without external CSS/JS.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
        ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src="$_cma_tgt" ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

DOC_DIR="${DOC_DIR:-$HOME/Documents}"
MD_FILE="${MD_FILE:-$DOC_DIR/Claude_Multi_Account_Fine_Tuning.md}"
HTML_FILE="${MD_FILE%.md}.html"
PDF_FILE="${MD_FILE%.md}.pdf"
DOCX_FILE="${MD_FILE%.md}.docx"
# Optional Word styling: first existing reference doc wins.
        Generate a starter
# with: pandoc -o reference.docx --print-default-data-file
        reference.docx
DOCX_REFERENCE="${DOCX_REFERENCE:-}"

[[ -f "$MD_FILE" ]] || cma_die "markdown not found: $MD_FILE"
cma_require pandoc

# --- Preprocess: expand `<!-- INCLUDE: relative/path -->` markers
        ---
# These markers, placed inside a fenced code block in the source
        markdown,
# get replaced with the contents of the referenced file (relative
        to the
# directory containing the markdown). That keeps the .md small
        while the
# generated HTML/PDF stays self-contained.
# Portable temp file: BSD/macOS mktemp has no --suffix. pandoc
        reads the
# format from --from, so the missing .md extension is irrelevant.
PROCESSED_MD="$(mktemp "${TMPDIR:-/tmp}/cma-export.XXXXXX")"
trap 'rm -f "$PROCESSED_MD"' EXIT

# Use only POSIX/2-arg awk match() (RSTART/RLENGTH + substr): the
        3-arg
# match($0,re,arr) capture form is a GNU-awk extension and breaks
        on the BSD
# awk that ships with macOS.
awk -v base="$(dirname "$MD_FILE")" '
  /<!-- INCLUDE: / {
    if (match($0, /INCLUDE: [^ ]+ -->/)) {
```

```
      tok = substr($0, RSTART, RLENGTH)   # "INCLUDE: path -->"
      sub(/^INCLUDE: /, "", tok)
      sub(/ -->$/, "", tok)               # -> "path"
      path = tok
      if (path !~ /^\//) path = base "/" path
      while ((getline line < path) > 0) print line
      close(path)
      next
    }
  }
  { print }
' "$MD_FILE" > "$PROCESSED_MD"

# --- HTML ---
# --standalone makes a complete HTML doc (head/body), --embed-
        resources
# inlines CSS/JS/images so the file is self-contained.
cma_log "rendering HTML -> $HTML_FILE"
pandoc \
  --from=gfm+yaml_metadata_block \
  --to=html5 \
  --standalone \
  --embed-resources \
  --toc --toc-depth=3 \
  --highlight-style=tango \
  --metadata title="Claude Multi-Account Fine Tuning" \
  -o "$HTML_FILE" \
  "$PROCESSED_MD"

# --- PDF ---
cma_log "rendering PDF -> $PDF_FILE"

render_pdf_via_pandoc_engine() {
  local engine="$1"
  command -v "$engine" >/dev/null 2>&1 || return 1
  pandoc \
    --from=gfm+yaml_metadata_block \
    --pdf-engine="$engine" \
    --toc --toc-depth=3 \
    --highlight-style=tango \
    -V geometry:margin=0.9in \
    -V mainfont="DejaVu Serif" \
    -V monofont="DejaVu Sans Mono" \
    --metadata title="Claude Multi-Account Fine Tuning" \
    -o "$PDF_FILE" \
    "$PROCESSED_MD"
}

render_pdf_via_chromium() {
  local engine
  for engine in chromium chromium-browser google-chrome chrome; do
    if command -v "$engine" >/dev/null 2>&1; then
      # Use the freshly generated HTML so styling matches.
```

```
      "$engine" --headless --no-sandbox --disable-gpu \
        --print-to-pdf="$PDF_FILE" \
        --print-to-pdf-no-header \
        "file://$HTML_FILE" >/dev/null 2>&1
      return $?
    fi
  done
  return 1
}

render_pdf_via_weasyprint_on_html() {
  command -v weasyprint >/dev/null 2>&1 || return 1
  weasyprint "$HTML_FILE" "$PDF_FILE"
}

# Try engines in order; the first that succeeds wins.
if render_pdf_via_pandoc_engine weasyprint 2>/dev/null; then
  cma_log "pdf engine: weasyprint (via pandoc)"
elif render_pdf_via_pandoc_engine wkhtmltopdf 2>/dev/null; then
  cma_log "pdf engine: wkhtmltopdf (via pandoc)"
elif render_pdf_via_weasyprint_on_html 2>/dev/null; then
  cma_log "pdf engine: weasyprint on html"
elif render_pdf_via_chromium 2>/dev/null; then
  cma_log "pdf engine: chromium headless"
else
  cma_die "no usable PDF engine. Install one of: weasyprint,
        wkhtmltopdf, chromium."
fi

# --- DOCX ---
# pandoc writes .docx natively (no external engine). A reference
        doc supplies
# styling (fonts, headings, code blocks) when provided.
cma_log "rendering DOCX -> $DOCX_FILE"
docx_args=(
  --from=gfm+yaml_metadata_block
  --to=docx
  --toc --toc-depth=3
  --highlight-style=tango
  --metadata title="Claude Multi-Account Fine Tuning"
  -o "$DOCX_FILE"
)
# Auto-discover a reference doc if none was passed explicitly.
if [[ -z "$DOCX_REFERENCE" ]]; then
  for cand in "$DOC_DIR/reference.docx" "$LIB_DIR/assets/
        reference.docx"; do
    [[ -f "$cand" ]] && { DOCX_REFERENCE="$cand"; break; }
  done
fi
[[ -n "$DOCX_REFERENCE" && -f "$DOCX_REFERENCE" ]] &&
        docx_args+=(--reference-doc "$DOCX_REFERENCE")
if pandoc "${docx_args[@]}" "$PROCESSED_MD"; then
```

```
  cma_log "docx ok${DOCX_REFERENCE:+ (styled via
        $DOCX_REFERENCE)}"
else
  cma_warn "DOCX render failed (pandoc); HTML + PDF are still
        produced"
fi

ls -lh "$HTML_FILE" "$PDF_FILE" "$DOCX_FILE" 2>/dev/null || ls
        -lh "$HTML_FILE" "$PDF_FILE"
```

8.8 install.sh

```
#!/usr/bin/env bash
# install.sh — One-shot bootstrap for the Claude multi-account
        toolkit.
# Idempotent; safe to re-run after pulling updates.
#
# What it does, in order:
#   1. Detect tooling (jq, rsync, awk, pandoc) and fail fast if
        missing.
#   2. Symlink every script into ~/.local/bin (creates the dir if
        needed).
#   3. Ensure ~/.local/bin is on PATH for future shells.
#   4. Create the managed alias file at $ALIAS_FILE and source it
        from
#      ~/.bashrc and ~/.zshrc (whichever exist).
#   5. Run claude-unify.sh to merge any existing account dirs.
#   6. Run claude-export-docs.sh to refresh the PDF/HTML alongside
        the
#      markdown doc.

set -euo pipefail

# macOS ships bash 3.2 which lacks `mapfile`. If a newer bash is
        available
# (Homebrew installs it at /opt/homebrew/bin/bash on Apple
        Silicon, or
# /usr/local/bin/bash on Intel), re-exec with it. Otherwise tell
        the user
# how to install it.
if (( BASH_VERSINFO[0] < 4 )); then
  for newer in /opt/homebrew/bin/bash /usr/local/bin/bash; do
    [[ -x "$newer" ]] && exec "$newer" "$0" "$@"
  done
  echo
        "claude-toolkit requires bash 4+. Install via: brew
        install bash" >&2
  exit 1
fi

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
        ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
```

```
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src="$_cma_tgt" ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

cma_log "platform: $(cma_os)"

# 1. Required tooling. pandoc is needed only for doc export; the
        rest are
# load-bearing for unify/add-account.
for t in rsync jq awk; do cma_require "$t"; done
command -v pandoc >/dev/null 2>&1 || cma_warn "pandoc not found —
        PDF/HTML export will be skipped"

# 2. Symlink the scripts onto PATH.
BIN_DIR="${BIN_DIR:-$HOME/.local/bin}"
mkdir -p "$BIN_DIR"
for f in "$LIB_DIR"/claude-*.sh; do
  name="$(basename "$f" .sh)"
  link="$BIN_DIR/$name"
  if [[ -L "$link" || -e "$link" ]]; then
    if [[ "$(readlink -f "$link" 2>/dev/null)" != "$(readlink -f
        "$f")" ]]; then
      mv "$link" "${link}.preunify.$(date +%Y%m%d%H%M%S)"
      ln -s "$f" "$link"
      cma_log "linked $link -> $f"
    fi
  else
    ln -s "$f" "$link"
    cma_log "linked $link -> $f"
  fi
done

# 3. Make sure ~/.local/bin is on PATH for new shells. We add
        to .bashrc
# and .zshrc once; existing shells need a manual reload.
PATH_LINE='export PATH="$HOME/.local/bin:$PATH"'
for rc in "${CMA_RC_FILES[@]}"; do
  [[ -f "$rc" ]] || continue
  if ! grep -F -q "$PATH_LINE" "$rc"; then
    printf '\n# Claude multi-account: ensure ~/.local/bin is on
        PATH\n%s\n' "$PATH_LINE" >> "$rc"
    cma_log "added PATH line to $rc"
  fi
done

# 4. Alias file + rc sourcing.
cma_ensure_alias_file
```

```
# Re-register any pre-existing aliases from $HOME/.bashrc into the
        managed
# alias file, then comment out the originals so they're not
        duplicated.
migrate_inline_aliases() {
  local rc="$1"
  [[ -f "$rc" ]] || return 0
  local tmp; tmp="$(mktemp)"
  local changed=0
  while IFS= read -r line; do
    if [[ "$line" =~ ^alias[[:space:]]+(claude[0-9a-zA-Z_-]
        +)=.*CLAUDE_CONFIG_DIR=([^[:space:]\"]+) ]]; then
      local a="${BASH_REMATCH[1]}" d="${BASH_REMATCH[2]}"
      d="${d//\"/}"
      d="${d/#\$HOME/$HOME}"
      d="${d/#~/$HOME}"
      cma_write_alias "$a" "$d"
      printf '# migrated to %s: %s\n' "$ALIAS_FILE" "$line" >>
        "$tmp"
      changed=1
    else
      printf '%s\n' "$line" >> "$tmp"
    fi
  done < "$rc"
  if (( changed )); then
    cp -p "$rc" "${rc}.preunify.$(date +%Y%m%d%H%M%S)"
    mv "$tmp" "$rc"
    cma_log "migrated inline claude* aliases in $rc ->
        $ALIAS_FILE"
  else
    rm -f "$tmp"
  fi
}
for rc in "${CMA_RC_FILES[@]}"; do
  migrate_inline_aliases "$rc"
done

# 4b. Copy proxy scripts for provider compatibility (e.g. Poe tool
        format fix).
PROXY_SRC="$LIB_DIR/proxy"
PROXY_DST="$SHARED_DIR/proxy"
if [[ -d "$PROXY_SRC" ]]; then
  mkdir -p "$PROXY_DST"
  cp "$PROXY_SRC"/*.py "$PROXY_DST/" 2>/dev/null && chmod +x
        "$PROXY_DST"/*.py 2>/dev/null
  cma_log "copied proxy scripts to $PROXY_DST"
fi

# 5. Unify whatever accounts exist now.
if (( $(cma_detect_accounts | wc -l) > 0 )); then
  cma_log "running claude-unify.sh"
  "$LIB_DIR/claude-unify.sh"
else
```

```
  cma_warn "no ~/${ACCOUNT_PREFIX}* dirs detected — add one with
        claude-add-account.sh"
fi

# 6. Refresh docs if pandoc is available.
if command -v pandoc >/dev/null 2>&1 && [[ -f "$HOME/Documents/
        Claude_Multi_Account_Fine_Tuning.md" ]]; then
  cma_log "running claude-export-docs.sh"
  "$LIB_DIR/claude-export-docs.sh" || cma_warn "doc export failed
        (continuing)"
fi

cat <<EOF

[done] claude-multi-account installed.

  Scripts on PATH:  $BIN_DIR/claude-{unify,add-account,remove-
        account,list-accounts,rollback,export-docs}
  Alias file:       $ALIAS_FILE
  Shared store:     $SHARED_DIR

Open a new shell (or run: source
        $ALIAS_FILE) so the aliases load, then:
  claude-list-accounts            # see what's wired up
  claude-add-account              # add another account
        interactively
EOF
```

8.9 tests/lib/assert.sh

```
#!/usr/bin/env bash
# assert.sh — small assertion helpers for the test suite. Each
        helper
# emits `[PASS]` or `[FAIL]` lines and increments counters in the
        caller.
# A test file should source this once and then call the helpers
        inside
# `it` blocks.

TESTS_PASSED="${TESTS_PASSED:-0}"
TESTS_FAILED="${TESTS_FAILED:-0}"
TESTS_FAILURES=()
CURRENT_TEST="${CURRENT_TEST:-(unknown)}"

# Mark the start of an individual test case. Use as:  it "merges
        history.jsonl"
it() {
  CURRENT_TEST="$*"
  printf '\n  • %s\n' "$CURRENT_TEST"
}

_pass() {
  TESTS_PASSED=$((TESTS_PASSED + 1))
```

```
  printf '    \033[32m[PASS]\033[0m %s\n' "$1"
}

_fail() {
  TESTS_FAILED=$((TESTS_FAILED + 1))
  TESTS_FAILURES+=("$CURRENT_TEST :: $1")
  printf '    \033[31m[FAIL]\033[0m %s\n' "$1"
  [[ -n "${2:-}" ]] && printf '           %s\n' "$2"
}

assert_eq() {
  local expected="$1" actual="$2" msg="${3:-equality}"
  if [[ "$expected" == "$actual" ]]; then _pass "$msg ($expected)"
  else _fail "$msg" "want=$expected got=$actual"; fi
}

assert_file() {
  local path="$1" msg="${2:-file exists}"
  if [[ -f "$path" ]]; then _pass "$msg: $path"
  else _fail "$msg" "missing: $path"; fi
}

assert_dir() {
  local path="$1" msg="${2:-dir exists}"
  if [[ -d "$path" ]]; then _pass "$msg: $path"
  else _fail "$msg" "missing dir: $path"; fi
}

assert_symlink_to() {
  local link="$1" expected="$2" msg="${3:-symlink target}"
  if [[ ! -L "$link" ]]; then
    _fail "$msg" "not a symlink: $link"
    return
  fi
  local actual; actual="$(readlink -f "$link")"
  local want;   want="$(readlink -f "$expected" 2>/dev/null ||
        echo "$expected")"
  if [[ "$actual" == "$want" ]]; then _pass "$msg: $link ->
        $expected"
  else _fail "$msg" "want=$want got=$actual"; fi
}

assert_not_symlink() {
  local p="$1" msg="${2:-not a symlink}"
  if [[ -L "$p" ]]; then _fail "$msg" "is a symlink: $p"
  else _pass "$msg: $p"; fi
}

assert_file_contains() {
  local path="$1" needle="$2" msg="${3:-file contains}"
  if grep -F -q -- "$needle" "$path" 2>/dev/null; then _pass
        "$msg: '$needle' in $(basename "$path")"
  else _fail "$msg" "'$needle' not found in $path"; fi
```

```
}

assert_file_not_contains() {
  local path="$1" needle="$2" msg="${3:-file lacks}"
  if grep -F -q -- "$needle" "$path" 2>/dev/null; then _fail
        "$msg" "'$needle' found in $path"
  else _pass "$msg: '$needle' absent from $(basename "$path")"; fi
}

assert_jq() {
  local path="$1" expr="$2" expected="$3" msg="${4:-jq check}"
  local actual; actual="$(jq -r "$expr" "$path" 2>/dev/null ||
        echo '<jq-error>')"
  if [[ "$actual" == "$expected" ]]; then _pass "$msg ($expr =
        $expected)"
  else _fail "$msg" "expr=$expr want=$expected got=$actual"; fi
}

assert_lines() {
  local path="$1" expected="$2" msg="${3:-line count}"
  local actual; actual="$(wc -l < "$path" 2>/dev/null || echo 0)"
  actual="${actual// /}"
  if [[ "$actual" == "$expected" ]]; then _pass "$msg ($expected
        lines)"
  else _fail "$msg" "want=$expected got=$actual lines in $path";
        fi
}

assert_exit() {
  local expected="$1"; shift
  local out; out="$("$@" 2>&1)"; local rc=$?
  if (( rc == expected )); then _pass "exit $expected for: $*"
  else _fail "exit code mismatch" "want=$expected got=$rc cmd=$*
        out=$out"; fi
}

summary() {
  echo
  if (( TESTS_FAILED == 0 )); then
    printf '\033[32m✓ %d passed, 0 failed\033[0m\n'
        "$TESTS_PASSED"
    return 0
  fi
  printf '\033[31m✗ %d failed, %d passed\033[0m\n'
        "$TESTS_FAILED" "$TESTS_PASSED"
  printf '\nFailures:\n'
  for f in "${TESTS_FAILURES[@]}"; do printf '  - %s\n' "$f"; done
  return 1
}
```

8.10 tests/lib/sandbox.sh

```
#!/usr/bin/env bash
# sandbox.sh — utilities for running scripts against a temp HOME
        so the
# real ~/.claude state is never touched.
#
# Usage from a test file:
#
#   source "$TESTS_DIR/lib/sandbox.sh"
#   make_sandbox          # creates $SANDBOX_HOME, exports HOME,
        etc.
#   make_account acct1    # creates a fake ~/.claude-acct1 inside
        sandbox
#   make_account acct2 --plugins
#   run_unify             # runs claude-unify.sh against the
        sandbox
#   cleanup_sandbox       # rm -rf the sandbox (registered with
        trap)

SCRIPTS_DIR="${SCRIPTS_DIR:-$HOME/Documents/scripts}"

make_sandbox() {
  SANDBOX_HOME="$(mktemp -d -t cma-test.XXXXXXXX)"
  # Override every env var the toolkit reads from. Pointing HOME
        at the
  # sandbox is the main switch; the other vars only matter if a
        caller
  # tests defaults explicitly.
  export HOME="$SANDBOX_HOME"
  export SHARED_DIR="$SANDBOX_HOME/.claude-shared"
  export ALIAS_FILE="$SANDBOX_HOME/.local/share/claude-multi-
        account/aliases.sh"
  export DEFAULT_DIR="$SANDBOX_HOME/.claude"
  export ACCOUNT_PREFIX=".claude-"
  export CLAUDE_BIN="/usr/bin/true"  # dummy; we never actually
        launch claude
  mkdir -p "$DEFAULT_DIR" "$SANDBOX_HOME/.local/bin"
  trap 'cleanup_sandbox' EXIT
}

cleanup_sandbox() {
  [[ -n "${SANDBOX_HOME:-}" && -d "$SANDBOX_HOME" ]] || return 0
  # Safety net: only delete if the path matches what mktemp would
        have
  # produced (cma-test.* somewhere under a tempdir root).
        Different
  # systems put mktemp output in different places (/tmp on most
        Linux,

        # /tmp/.private/<user>/ on this host, /var/folders/... on
        macOS) so
  # we anchor on the cma-test. prefix rather than the parent.
  case "$(basename "$SANDBOX_HOME")" in
```

```
    cma-test.*) rm -rf -- "$SANDBOX_HOME" ;;
    *) echo "refusing to rm sandbox at unexpected path:
        $SANDBOX_HOME" >&2 ;;
  esac
}

# make_account NAME [--plugins] [--settings JSON] [--history
        "line1|line2"] [--memory KEY:VALUE...]
make_account() {
  local name="$1" with_plugins=0 settings_json="" history_lines=()
  shift
  local memory_entries=() todo_entries=()
  while (( $# )); do
    case "$1" in
      --plugins) with_plugins=1; shift ;;
      --settings) settings_json="$2"; shift 2 ;;
      --history) IFS='|' read -r -a history_lines <<< "$2"; shift
        2 ;;
      --memory)  memory_entries+=("$2"); shift 2 ;;
      --todo)    todo_entries+=("$2"); shift 2 ;;
      *) echo "make_account: unknown arg $1" >&2; return 1 ;;
    esac
  done

  local dir="$HOME/${ACCOUNT_PREFIX}${name}"
  mkdir -p "$dir/projects/-home-test/memory" "$dir/todos" "$dir/
        tasks" \
           "$dir/plans" "$dir/file-history" "$dir/paste-cache" \
           "$dir/shell-snapshots" "$dir/session-env" "$dir/
        telemetry" \
           "$dir/sessions" "$dir/backups" "$dir/cache" "$dir/
        plugins"

  # Fake credentials so list-accounts reports CREDS:yes.
  printf '{"account":"%s"}\n' "$name" > "$dir/.credentials.json"
  printf '{"name":"%s"}\n' "$name" > "$dir/.claude.json"

  # A unique transcript so the union-merge step has something to
        merge.
  printf '{"role":"user","msg":"hi from %s"}\n' "$name" \
    > "$dir/projects/-home-test/$(uuidgen 2>/dev/null || echo
        "session-${name}").jsonl"

  # Memory + todo content if requested.
  local m
  for m in "${memory_entries[@]}"; do
    local k="${m%%:*}" v="${m#*:}"
    printf -- '%s\n' "$v" > "$dir/projects/-home-test/memory/$
        {k}.md"
  done
  local t
  for t in "${todo_entries[@]}"; do
    printf '{"subject":"%s"}\n' "$t" > "$dir/todos/$(uuidgen 2>/
        dev/null || echo "todo-${name}-$t").json"
```

```
  done

  # History lines (used to test concat+dedup later).
  if (( ${#history_lines[@]} )); then
    printf '%s\n' "${history_lines[@]}" > "$dir/history.jsonl"
  fi

  # Settings JSON if specified, else a minimal default.
  if [[ -n "$settings_json" ]]; then
    printf '%s\n' "$settings_json" > "$dir/settings.json"
  else
    printf '{"enabledPlugins":{"%s-plugin":true}}\n' "$name" >
        "$dir/settings.json"
  fi

  if (( with_plugins )); then
    mkdir -p "$dir/plugins/cache/test-marketplace/test-plugin/
        1.0.0"
    mkdir -p "$dir/plugins/marketplaces/test-marketplace"
    cat > "$dir/plugins/installed_plugins.json" <<EOF
{
  "version": 2,
  "plugins": {
    "test-plugin@test-marketplace": [
      {
        "scope": "project",
        "installPath": "$dir/plugins/cache/test-marketplace/test-
        plugin/1.0.0",
        "version": "1.0.0"
      }
    ]
  }
}
EOF
    cat > "$dir/plugins/known_marketplaces.json" <<EOF
{
  "test-marketplace": {
    "source": {"source": "github", "repo": "example/test"},
    "installLocation": "$dir/plugins/marketplaces/test-
        marketplace"
  }
}
EOF
  fi

  echo "$dir"
}

run_unify() {
  "$SCRIPTS_DIR/claude-unify.sh" "$@"
}

run_add_account() {
```

```
  "$SCRIPTS_DIR/claude-add-account.sh" "$@"
}

run_remove_account() {
  "$SCRIPTS_DIR/claude-remove-account.sh" "$@"
}

run_list_accounts() {
  "$SCRIPTS_DIR/claude-list-accounts.sh" "$@"
}

run_rollback() {
  "$SCRIPTS_DIR/claude-rollback.sh" "$@"
}
```

8.11 tests/run-all.sh

```
#!/usr/bin/env bash
# run-all.sh — discover every test_*.sh sibling, run each in its
        own
# subshell, and tally pass/fail.
#
# Exit code is 0 only if every test file's `summary` returned 0.

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"
export SCRIPTS_DIR

FILES=()
if (( $# )); then
  for arg in "$@"; do FILES+=("$TESTS_DIR/test_${arg}.sh"); done
else
  mapfile -t FILES < <(find "$TESTS_DIR" -maxdepth 1 -name
        'test_*.sh' -type f | sort)
fi

PASSED=0 FAILED=0 FAILED_FILES=()

for f in "${FILES[@]}"; do
  [[ -f "$f" ]] || { echo "missing: $f" >&2; FAILED=$
        ((FAILED+1)); FAILED_FILES+=("$f"); continue; }
  printf '\n\033[1m==> %s\033[0m\n' "$(basename "$f")"
  if bash "$f"; then
    PASSED=$((PASSED + 1))
  else
    FAILED=$((FAILED + 1))
    FAILED_FILES+=("$(basename "$f")")
  fi
done
```

```
echo
echo "========================================="
echo "Test files: $((PASSED + FAILED))   passed: $PASSED
        failed: $FAILED"
if (( FAILED )); then
  echo "Failed files:"
  for f in "${FAILED_FILES[@]}"; do echo "  - $f"; done
  exit 1
fi
echo "ALL GREEN"
```

8.12 tests/test_lib.sh

```
#!/usr/bin/env bash
# test_lib.sh — unit tests for the helper functions in lib.sh.
set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

# shellcheck source=lib/assert.sh
source "$TESTS_DIR/lib/assert.sh"
# shellcheck source=lib/sandbox.sh
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox
# shellcheck source=../lib.sh
source "$SCRIPTS_DIR/lib.sh"
# lib.sh enables `set -e`. The test harness is intentionally
        tolerant of
# non-zero exits (we assert on them), so turn it back off here.
set +e

it "cma_validate_alias accepts well-formed names"
( set -e; cma_validate_alias "claude3" ); assert_eq 0 $? "claude3"
( set -e; cma_validate_alias "work_acct-1" ); assert_eq 0 $?
        "work_acct-1"

it "cma_validate_alias rejects invalid names"
( cma_validate_alias "3claude" >/dev/null 2>&1 ); assert_eq 1 $?
        "rejects digit-leading"
( cma_validate_alias "bad name" >/dev/null 2>&1 ); assert_eq 1 $?
        "rejects spaces"
( cma_validate_alias "" >/dev/null 2>&1 ); assert_eq 1 $?
        "rejects empty"

it "cma_suggest_alias starts at claude1 when nothing exists"
suggestion="$(cma_suggest_alias)"
assert_eq "claude1" "$suggestion" "first suggestion"

it "cma_suggest_alias increments past existing claudeN aliases"
cma_write_alias claude1 "$HOME/.claude-acct1"
cma_write_alias claude2 "$HOME/.claude-acct2"
```

```
cma_write_alias claude5 "$HOME/.claude-acct5"   # gap
suggestion="$(cma_suggest_alias)"
assert_eq "claude6" "$suggestion" "skips past highest, not count"

it "cma_write_alias is idempotent — rewriting same alias doesn't
        duplicate"
cma_write_alias claude1 "$HOME/.claude-acct1"   # second write
count="$(grep -c '^alias claude1=' "$ALIAS_FILE")"
assert_eq "1" "$count" "one alias line for claude1"

it "cma_remove_alias removes the line"
cma_remove_alias claude5
assert_file_not_contains "$ALIAS_FILE" "alias claude5=" "claude5
        removed"

it "cma_ensure_alias_file sources from the shell rc file"
# lib.sh manages .zshrc on macOS and .bashrc + .zshrc on Linux
        (CMA_RC_FILES).
# Assert against the platform-appropriate target, selected the
        same way lib.sh
# selects it, so the test is correct on both OSes.
if [[ "$(uname -s)" == "Darwin" ]]; then RC_FILE="$HOME/.zshrc";
        else RC_FILE="$HOME/.bashrc"; fi
touch "$RC_FILE"
rm -f "$ALIAS_FILE"
cma_ensure_alias_file
assert_file "$ALIAS_FILE" "alias file created"
assert_file_contains "$RC_FILE" "source \"$ALIAS_FILE\"" "rc file
        gets source line"

it "cma_detect_accounts skips the shared store"
mkdir -p "$HOME/.claude-shared" "$HOME/.claude-acct1"
mapfile -t found < <(cma_detect_accounts)
joined="${found[*]}"
[[ "$joined" == *".claude-acct1"* ]]; assert_eq 0 $?
        "finds .claude-acct1"
[[ "$joined" != *".claude-shared"* ]]; assert_eq 0 $?
        "excludes .claude-shared"

it "cma_detect_accounts excludes non-Claude .claude-* dirs
        (e.g. .claude-server-commander)"
# Mimic the real-world false positive seen on mistborn.local: an
        MCP server
# config dir whose name happens to start with .claude- but has
        only its own
# config files, no Claude markers.
mkdir -p "$HOME/.claude-server-commander"
printf '{}\n' > "$HOME/.claude-server-commander/config.json"
printf '{}\n' > "$HOME/.claude-server-commander/feature-
        flags.json"
mapfile -t found < <(cma_detect_accounts)
joined="${found[*]}"
[[ "$joined" != *".claude-server-commander"* ]]; assert_eq 0 $?
        "excludes .claude-server-commander"
```

```
[[ "$joined" == *".claude-acct1"* ]]; assert_eq 0 $? "still finds
        the legit empty account"

it "cma_detect_accounts includes a populated account dir even if
        it has foreign config too"
# A real account dir with the Claude marker (projects/) shouldn't
        get
# falsely excluded just because some other file happens to be
        there.
mkdir -p "$HOME/.claude-real/projects"
printf '{}\n' > "$HOME/.claude-real/some-other-tool.json"
mapfile -t found < <(cma_detect_accounts)
joined="${found[*]}"
[[ "$joined" == *".claude-real"* ]]; assert_eq 0 $?
        "finds .claude-real"

summary
```

8.13 tests/test_unify.sh

```
#!/usr/bin/env bash
# test_unify.sh — exercises claude-unify.sh:
#   * basic merge of two accounts: all shared items become
        symlinks
#   * settings.json enabledPlugins is a strict union
#   * history.jsonl is concatenated and de-duplicated
#   * memory files survive merge (newest account wins conflicts)
#   * plugin manifest installPath is rewritten to the shared store
#   * N>2 accounts also work
#   * a second unify run produces the same state (idempotent)
#   * --rollback restores .preunify.* backups

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox

# Two fixture accounts with overlapping and unique data.
acct1="$(make_account acct1 --plugins \
  --settings '{"enabledPlugins":
        {"a":true,"b":true},"effortLevel":"high"}' \
  --history 'cmd1|cmd2|shared-line' \
  --memory 'user_role:role from acct1' \
  --todo todo-a-1)"
acct2="$(make_account acct2 --plugins \
  --settings '{"enabledPlugins":
        {"b":true,"c":true},"skipDangerousModePermissionPrompt":true}'
        \
```

```
  --history 'cmd3|shared-line|cmd4' \
  --memory 'user_role:role from acct2 (NEWER)' \
  --memory 'project_notes:notes from acct2 only' \
  --todo todo-b-1)"

# Make sure acct2 is the most recently touched so the unifier's "last
# account wins on conflicts" rule applies to it.
touch "$acct2/projects/-home-test/memory/user_role.md"

run_unify >/dev/null 2>&1
rc=$?
assert_eq 0 "$rc" "claude-unify exits 0"

it "every shared item becomes a symlink into the shared store"
for item in projects todos tasks plans file-history paste-cache plugins \
            shell-snapshots session-env telemetry sessions backups cache \
            stats-cache.json history.jsonl settings.json; do
  case "$item" in
    stats-cache.json)
      # Only fixtures with this file get the symlink; skip if absent.
      [[ -e "$SHARED_DIR/$item" ]] || continue ;;
  esac
  assert_symlink_to "$acct1/$item" "$SHARED_DIR/$item" "$acct1 $item linked"
  assert_symlink_to "$acct2/$item" "$SHARED_DIR/$item" "$acct2 $item linked"
done

it "settings.json enabledPlugins is the union {a,b,c}"
assert_jq "$SHARED_DIR/settings.json" '.enabledPlugins | keys | sort | join(",")' "a,b,c" "union"
assert_jq "$SHARED_DIR/settings.json" '.effortLevel' "high" "effortLevel preserved"
assert_jq "$SHARED_DIR/settings.json" '.skipDangerousModePermissionPrompt' "true" "permission flag preserved"

it "history.jsonl contains every unique line, no duplicates"
expected_lines=5
actual_lines="$(sort -u "$SHARED_DIR/history.jsonl" | wc -l | tr -d ' ')"
assert_eq "$expected_lines" "$actual_lines" "5 unique lines"
assert_file_contains "$SHARED_DIR/history.jsonl" "cmd1" "cmd1 present"
assert_file_contains "$SHARED_DIR/history.jsonl" "cmd4" "cmd4 present"

it "memory: newest account's version wins on filename conflict"
assert_file_contains "$SHARED_DIR/projects/-home-test/memory/user_role.md" "NEWER" "acct2 wins"
```

```
assert_file "$SHARED_DIR/projects/-home-test/memory/
        project_notes.md" "acct2-only memory survives"

it "plugin manifest paths are rewritten to the shared store"
assert_jq "$SHARED_DIR/plugins/installed_plugins.json" \
  '.plugins["test-plugin@test-marketplace"][0].installPath' \
  "$SHARED_DIR/plugins/cache/test-marketplace/test-plugin/1.0.0" \
  "installPath rewritten"
assert_jq "$SHARED_DIR/plugins/known_marketplaces.json" \
  '.["test-marketplace"].installLocation' \
  "$SHARED_DIR/plugins/marketplaces/test-marketplace" \
  "installLocation rewritten"

it "credentials are NOT shared — each account keeps its own"
assert_not_symlink "$acct1/.credentials.json" "acct1 creds local"
assert_not_symlink "$acct2/.credentials.json" "acct2 creds local"
assert_file_contains "$acct1/.credentials.json" "acct1" "acct1
        creds intact"
assert_file_contains "$acct2/.credentials.json" "acct2" "acct2
        creds intact"

it "re-running claude-unify is idempotent (settings.json byte-
        stable)"
checksum_before="$(sha256sum "$SHARED_DIR/settings.json" | cut
        -d' ' -f1)"
target_before="$(readlink -f "$acct1/projects")"
run_unify >/dev/null 2>&1
checksum_after="$(sha256sum "$SHARED_DIR/settings.json" | cut -d'
        ' -f1)"
target_after="$(readlink -f "$acct1/projects")"
assert_eq "$checksum_before" "$checksum_after" "settings.json
        unchanged"
assert_eq "$target_before"   "$target_after"   "symlink target
        unchanged"

it "N>2 accounts: adding a third and re-running unifies it too"
acct3="$(make_account acct3 \
  --settings '{"enabledPlugins":{"d":true}}' \
  --history 'cmd5|cmd1')"
run_unify >/dev/null 2>&1
assert_symlink_to "$acct3/projects" "$SHARED_DIR/projects" "acct3
        projects linked"
assert_jq "$SHARED_DIR/settings.json" '.enabledPlugins.d' "true"
        "acct3 plugin merged"
assert_file_contains "$SHARED_DIR/history.jsonl" "cmd5" "acct3
        history merged"

it "--rollback restores backups and archives the shared store"
run_rollback >/dev/null 2>&1
assert_dir "$acct1/projects" "acct1 projects restored as real dir"
assert_not_symlink "$acct1/projects" "no longer a symlink"
[[ ! -d "$SHARED_DIR" ]]; assert_eq 0 $? "shared store moved
        aside"
shared_archive="$(find "$HOME" -maxdepth 1 -name '.claude-
        shared.removed.*' -type d 2>/dev/null | head -1)"
```

```
[[ -n "$shared_archive" ]]; assert_eq 0 $? "archive sibling
        exists"

summary
```

8.14 tests/test_add_remove.sh

```
#!/usr/bin/env bash
# test_add_remove.sh — exercises claude-add-account.sh and
# claude-remove-account.sh end-to-end:
#   * add suggests the next free claudeN
#   * add creates the dir with the right symlinks
#   * add writes a shell-safe alias line
#   * add refuses to overwrite an existing dir
#   * add accepts a custom alias name
#   * remove drops the alias line
#   * remove archives the dir by default, --delete removes it

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox

# Start with two existing accounts and a unified shared store.
acct1="$(make_account acct1)"
acct2="$(make_account acct2)"
run_unify >/dev/null 2>&1

it "claude-add-account --yes uses sensible defaults"
run_add_account --alias claude3 --yes >/dev/null 2>&1
rc=$?
assert_eq 0 "$rc" "exit 0"
new_dir="$HOME/.claude-claude3"
assert_dir "$new_dir" "config dir created"
assert_symlink_to "$new_dir/projects" "$SHARED_DIR/projects"
        "projects linked"
assert_symlink_to "$new_dir/settings.json" "$SHARED_DIR/
        settings.json" "settings linked"
assert_symlink_to "$new_dir/CLAUDE.md" "$SHARED_DIR/CLAUDE.md"
        "CLAUDE.md linked"
assert_file_contains "$ALIAS_FILE" "alias claude3=" "alias line
        written"
assert_file_contains "$ALIAS_FILE" "CLAUDE_CONFIG_DIR=$new_dir"
        "alias points at new dir"

it "claude-add-account refuses to overwrite an existing dir"
( run_add_account --alias claude3 --yes >/dev/null 2>&1 )
rc=$?
```

```
[[ $rc -ne 0 ]]; assert_eq 0 $? "exits non-zero when dir exists"

it "claude-add-account accepts a custom alias name and custom dir"
custom_dir="$HOME/.claude-mywork"
run_add_account --alias mywork --dir "$custom_dir" --yes >/dev/
        null 2>&1
assert_dir "$custom_dir" "custom dir created"
assert_symlink_to "$custom_dir/projects" "$SHARED_DIR/projects"
        "projects linked"
assert_file_contains "$ALIAS_FILE" "alias mywork=" "custom alias
        written"

it "claude-add-account validates alias names"
( run_add_account --alias "bad name" --yes >/dev/null 2>&1 )
rc=$?
[[ $rc -ne 0 ]]; assert_eq 0 $? "rejects bad alias"

it "claude-remove-account --archive moves the dir aside"
run_remove_account --alias claude3 --archive --yes >/dev/null 2>&1
rc=$?
assert_eq 0 "$rc" "exit 0"
[[ ! -d "$HOME/.claude-claude3" ]]; assert_eq 0 $? "original dir
        gone"
archived="$(find "$HOME" -maxdepth 1 -name '.claude-
        claude3.removed.*' -type d 2>/dev/null | head -1)"
[[ -n "$archived" ]]; assert_eq 0 $? "archived sibling exists"
assert_file_not_contains "$ALIAS_FILE" "alias claude3=" "alias
        line removed"

it "claude-remove-account --delete actually deletes"
run_remove_account --alias mywork --delete --yes >/dev/null 2>&1
[[ ! -d "$HOME/.claude-mywork" ]]; assert_eq 0 $? "dir deleted"
[[ -z "$(find "$HOME" -maxdepth 1 -name '.claude-
        mywork.removed.*' 2>/dev/null)" ]]; assert_eq 0 $? "no
        archive sibling"

it "claude-remove-account rejects unknown alias"
( run_remove_account --alias nonexistent --yes >/dev/null 2>&1 )
rc=$?
[[ $rc -ne 0 ]]; assert_eq 0 $? "unknown alias errors out"

summary
```

8.15 tests/test_list.sh

```
#!/usr/bin/env bash
# test_list.sh — claude-list-accounts.sh reports accurate state.

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"
```

```
source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox

# Empty-host case first.
it "lists nothing when no accounts exist"
out="$(run_list_accounts 2>&1)"
[[ "$out" == *"ALIAS"* ]]; assert_eq 0 $? "header always printed"
echo "$out" | grep -E '^\-' >/dev/null
[[ $? -ne 0 ]]; assert_eq 0 $? "no account rows"

acct1="$(make_account acct1)"
acct2="$(make_account acct2)"
run_unify >/dev/null 2>&1
run_add_account --alias claude3 --yes >/dev/null 2>&1

it "shows every account dir with creds + link status"
out="$(run_list_accounts 2>&1)"
assert_file_contains <(printf '%s\n' "$out") ".claude-acct1"
        "acct1 listed"
assert_file_contains <(printf '%s\n' "$out") ".claude-acct2"
        "acct2 listed"
assert_file_contains <(printf '%s\n' "$out") ".claude-claude3"
        "claude3 listed"
assert_file_contains <(printf '%s\n' "$out") "claude3" "claude3
        alias resolved"

it "reports the shared store path"
out="$(run_list_accounts 2>&1)"
assert_file_contains <(printf '%s\n' "$out") "$SHARED_DIR"
        "shared store path printed"

it "flags accounts with missing credentials"
# claude3 was created by add-account; it has no .credentials.json.
out="$(run_list_accounts 2>&1)"
line="$(echo "$out" | grep ".claude-claude3" || true)"
[[ "$line" == *" no "* ]]; assert_eq 0 $? "claude3 creds:no"
line1="$(echo "$out" | grep ".claude-acct1" || true)"
[[ "$line1" == *" yes "* ]]; assert_eq 0 $? "acct1 creds:yes"

summary
```

8.16 tests/test_export.sh

```
#!/usr/bin/env bash
# test_export.sh — claude-export-docs.sh produces HTML and PDF,
        expands
# <!-- INCLUDE: path --> markers, and the output PDF starts with
        "%PDF-".

set -uo pipefail
```

```
TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox
mkdir -p "$HOME/Documents/scripts"

# Tiny fixture markdown + a fixture script to include. We point
        the
# export script at this synthetic doc instead of the real one.
fixture_script="$HOME/Documents/scripts/sample.sh"
cat > "$fixture_script" <<'EOF'
#!/usr/bin/env bash
echo "SAMPLE_SCRIPT_BODY_MARKER"
EOF

fixture_md="$HOME/Documents/fixture.md"
cat > "$fixture_md" <<'EOF'
---
title: Fixture
---
# Fixture

Body paragraph above the include.

```bash
<!-- INCLUDE: scripts/sample.sh -->
```

End of fixture. EOF

it "MD_FILE env override is honored and HTML is generated" MD_FILE="$fixture_md" "$SCRIPTS_DIR/claude-export-docs.sh" >/dev/null 2>&1 rc=$? assert_eq 0 "$rc" "export exits 0" assert_file "$HOME/Documents/fixture.html" "html generated" assert_file "$HOME/Documents/fixture.pdf" "pdf generated"

it " markers are expanded in output" assert_file_contains "$HOME/Documents/fixture.html" "SAMPLE_SCRIPT_BODY_MARKER" "include expanded" assert_file_not_contains "$HOME/Documents/fixture.html" "<!-- INCLUDE:" "markers removed"

it "PDF starts with the %PDF- magic bytes" head -c 5 "$HOME/Documents/fixture.pdf" | grep -q '^%PDF-' assert_eq 0 $? "PDF magic header present"

it "regeneration is non-destructive (file timestamps move forward)" MD_FILE="$fixture_md" "$SCRIPTS_DIR/claude-export-docs.sh" >/dev/null 2>&1 assert_file "$HOME/Documents/fixture.html" "html still present after rerun" assert_file "$HOME/Documents/fixture.pdf" "pdf still present after rerun"
```

summary

```
---

## 9. Troubleshooting

| Symptom                                                          |
Likely cause                                              |
Fix
|
| ---------------------------------------------------------------- |
---------------------------------------------------------- |
-----------------------------------------------------------------------
|
| `claude1` works, `claude2` shows empty memory                    |
Symlinks weren't created in `~/.claude-milos85vasic2nd/`.   | Re-
run `claude-unify.sh` — it's idempotent.
|
| New account can't see history                                    |
Forgot to reload shell after `claude-add-account.sh`.       |
`exec $SHELL -l` or `source ~/.local/share/claude-multi-account/
aliases.sh`. |
| Plugin loads fail with "plugin not found"                        |
Manifest path points outside the shared store.              | Re-
run `claude-unify.sh` — the path-rewrite pass will fix it.
|
| Both accounts see *each other's* drafts in the queue             |
`sessions/` is shared and two processes are running at once. |
Don't run two accounts concurrently, or remove `sessions` from
`SHARED_ITEMS`. |
| `pandoc` fails to find a PDF engine                              |
No weasyprint/wkhtmltopdf/chromium installed.               |
`brew install weasyprint` or `apt install
weasyprint`.               |
| Want to start completely over                                    |
-                                                           |
`claude-rollback` then re-run
`install.sh`.                           |

## 10. Sharing the ecosystem with OpenCode

The same plugin ecosystem that the multi-account setup unifies can
also be
shared with a host-installed [OpenCode](https://opencode.ai).
Claude Code
plugins are runtime-specific (hooks + JS), but their portable
contents —
Anthropic-format **Skills** (`SKILL.md`), **MCP servers**
(`.mcp.json`), and
the user **`CLAUDE.md`** — map directly onto OpenCode's native
`skills.paths`,
`mcp{}`, and `instructions[]` config keys.
```

```
`claude-opencode-sync` performs this translation in one
idempotent, additive
pass (it never clobbers OpenCode's own providers or servers):

```bash
claude-opencode-sync --dry-run --stats   # preview
claude-opencode-sync                      # apply (prior config is
backed up)
```

It scans the plugin cache, parses both the wrapped (`{"mcpServers":{…}}`) and bare (`{name:{…}}`) `.mcp.json` shapes, expands `${CLAUDE_PLUGIN_ROOT}`, deduplicates servers shipped by multiple plugins, and applies a conservative enable policy — enabling only no-auth/no-secret servers by default so OpenCode startup stays fast, while configuring all the rest `enabled:false`, ready to `opencode mcp auth`. On the reference host this exposes 1,000+ skills and 110+ MCP servers to OpenCode.

End-to-end proof (sandbox suite + live verification against the real OpenCode binary, writing inspectable evidence to `scripts/tests/proof/`):

```bash
bash scripts/tests/run-proof.sh
```

Full guide, architecture diagrams, and the enable-policy flowchart live in `OpenCode_Integration.md` and `docs/diagrams/`.

---

11. Provider aliases — running other LLMs through Claude Code

The same alias machinery that powers `claude1..N` also drives **provider aliases**: a Claude Code alias per LLM provider whose key you keep in your keys file, pointed at that provider's strongest model. The command is `claude-providers` and it is **fully dynamic** — no provider, base URL, or model ID is hardcoded.

Data flow (nothing hardcoded)

1. `~/api_keys.sh` provides the API-key *variable names* (read by `grep`, never executed).
2. `models.dev/api.json` (fetched + cached, with graceful offline degrade) provides each provider's endpoint, transport hint (`npm`), and full model catalog with cost/context/reasoning metadata.
3. `providers_resolve.py` resolves each key into a record: provider id, alias, base URL, transport (`native` iff the provider speaks the Anthropic API, else `router`), and a strong + fast model chosen from the metadata.
```

4. The bundled `LLMsVerifier` submodule (or a lightweight HTTP probe) optionally verifies the key + model before the alias is activated.
5. For each provider: a non-secret env file, a `cma_run_provider <id>` alias, a `~/.claude-prov-<id>` config dir linking all shared items (so every plugin is available), and the always-on plugin set are generated.

Two tiny editable inputs cover the only things data can't infer: `providers/key-aliases.json` (key-name → provider normalization) and `providers/overrides.json` (per-provider pins, e.g. a short `dseek` alias or a native `/anthropic` endpoint).

Transports

- **native** — `claude` runs directly with `ANTHROPIC_BASE_URL`, `ANTHROPIC_AUTH_TOKEN`, `ANTHROPIC_MODEL`, `ANTHROPIC_SMALL_FAST_MODEL`.
- **router** — launches through claude-code-router (`ccr code`), which translates Anthropic ⇄ OpenAI/Gemini. The provider entry is written into the ccr config with the live key at launch (chmod 600), so the toolkit itself never persists secrets.

Safety

Provider config dirs are **excluded from account auto-detection**, so they never merge into real-account auth and never disturb `claude-unify` / `claude-add-account`. Existing `claudeN` accounts keep working unchanged, and new accounts are still created the same way.

Commands

```
claude-providers sync          # discover + create/refresh every
        provider alias
claude-providers list          # alias, provider, transport,
        strong/fast model
claude-providers show <id>     # one provider's resolved env
claude-providers remove <id>   # remove alias + env, back up the
        config dir
claude-providers add --from-key VAR --id PROVIDER   # register a
        mapping, then sync
```

Known limitation — session color

The goal of defaulting each provider session's `/color` to purple is **not automatable** on the installed Claude Code (v2.1.178): `/color` is session-scoped and TUI-only, never persisted, with no settings key or env var. Type `/color purple` per session. (Documented in `docs/Provider_Aliases_User_Guide.md`.)

Full details, overrides, verification, and troubleshooting live in `docs/Provider_Aliases_User_Guide.md`; diagrams in `docs/diagrams/provider-aliases.md`.

Generated 2026-05-26 (OpenCode integration added 2026-06-06; provider aliases added 2026-06-16). Maintain by editing this markdown file and re-running `claude-export-docs.sh`.